Project Report On

AI-Powered Code assistant with RAG and Cross-language Support

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

By

Aleena Althaf (U2203217)
Anithra Syindo (U2203044)
Benita Benny (U2203065)
Christine Michael (U2203070)

Under the guidance of

Dr. Anita John

Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039
March 2026

CERTIFICATE

This is to certify that the project report entitled ***"AI-Powered Code assistant with RAG and Cross-language Support"*** *is a bonafide record of the work done by* ***Aleena Althaf (U2203217), Anithra Syindo(U2203044), Benita Benny (U2203065), Christine Michael (U2203070)****, submitted to Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in "Computer Science and Engineering" during the academic year 2025-2026.*

Project Guide
Dr. Anita John
Associate Professor
Dept. of CSE
RSET

Project Coordinator
Dr. Jisha G
Associate Professor
Dept. of CSE
RSET

Head of the Department
Dr. Preetha K G
Professor
Dept. of CSE
RSET

ACKNOWLEDGMENT

We wish to express our sincere gratitude towards **Rev. Dr. Jaison Paul Mulerrikal CMI**, Principal of RSET, and **Dr. Preetha K G**, Head of the Department of Computer Science and Engineering for providing us with the opportunity to undertake our project, "AI-Powered Code assistant with RAG and Cross-language Support".

We are highly indebted to our project coordinators, **Dr. Jisha G**, Associate Professor, Department of Computer Science and Engineering and **Dr. Preetha K G**, Professor and Head of the Department of Computer Science and Engineering, for their valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Dr. Anita John**, Associate Professor, Department of Computer Science and Engineering, for her patience and all the priceless advice and wisdom she has shared with us.

Last but not least, we would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Aleena Althaf
Anithra Syindo
Benita Benny
Christine Michael

Abstract

This project presents an AI-based Code Assistant designed to help users understand and improve programming code more effectively. The system integrates Retrieval-Augmented Generation (RAG) with Large Language Models to provide intelligent support for common programming tasks. It addresses issues like comprehending unfamiliar code, identifying compilation errors, optimizing programs, and translating code between selected programming languages.

It supports features like syntax checking, code optimization, code conversion between Python and Java, line-by-line code explanation, and automatic generation of algorithms and flowcharts from problem statements.

The system makes use of technologies like Sentence Transformers for embeddings, CodeT5 for code translation, ChromaDB for vector retrieval, and LLaMA models via the Groq API for the generation of algorithms and explanations. This combination of features allows users to grasp code more easily, spot mistakes, and ultimately, boost the quality of their programs, all within a unified platform.

Contents

Acknowledgment **i**

Abstract **ii**

List of Abbreviations **vi**

List of Figures **vii**

List of Tables **viii**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 1

1.3 Scope and Motivation 1

1.4 Project Objectives 2

1.5 Risks and Challenges 2

1.6 Assumptions 3

1.7 Societal/Industrial Relevance 4

1.8 Organization of the Report 4

1.9 Summary of the Chapter 6

2 Literature Survey **7**

2.1 Y. Rong, T. Du, R. Li, W. Bao (2025). Integrating LLM-Based Code Optimization with Human-Like Exclusionary Reasoning for Computational Education[1] 7

2.2 Chunyan Zhang, Qinglei Zhou, Meng Qiao(2022).Re_Trans: Combined Retrieval and Transformer Model for Source Code Summarization[2] 8

2.3 Yin, X., Ni, C., Nguyen, T. N., Wang, S., & Yang, X. (2024). Rectifier: Code Translation with Corrector via LLMs[3] 8

2.4 R. Zhang, H. Li, Z. Liu, and J. Yang.(2024).READSUM: Retrieval-Augmented Adaptive Transformer for Source Code Summarization[4] 9
2.5 Comparison and Gaps Identified 11
2.6 Research Gaps Identified 12
2.7 Summary of the Chapter 12

3 System Design **13**
3.1 Overall System Architecture 13
3.2 Design Diagrams 15
3.3 Component Design 16
3.4 Algorithms Used and Design 17
3.5 Software and Hardware Requirements 18
3.6 Budget 18
3.7 Module Division and Work Responsibilities 18
3.8 Key Deliverables - Expected Outputs 23

4 System Implementation **25**
4.1 Dataset Preparation and Processing 25
4.2 Syntax Checker Implementation 26
4.3 Code Optimization Implementation 26
4.4 Code Conversion Module 27
4.5 Code Explanation Module (RAG-Based) 27
4.6 Algorithm and Flowchart Generation 28
4.7 Summary of the Chapter 29

5 Results and Discussion **30**
5.1 System Implementation Results 30

6 Conclusion & Future Scope **38**
6.1 Future Scope 39

References **40**

Appendix A: Presentation **41**

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes 68

Appendix C: CO-PO-PSO Mapping 73

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
CFG	Control Flow Graph
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
NLP	Natural Language Processing
ML	Machine Learning
IDE	Integrated Development Environment
k-NN	k-Nearest Neighbour
JSON	JavaScript Object Notation

List of Figures

3.1 Architecture Diagram 21
3.2 Sequence Diagram 22
3.3 Gantt Chart 24

5.1 Static Syntax Checker 31
5.2 Code Optimization Feature 32
5.3 Input Code For Explanation 32
5.4 Code Explanation Output 33
5.5 Prompt For Algorithm Generation 34
5.6 Generated Algorithm 34
5.7 Flowchart Prompt 35
5.8 Flowchart Generated 36
5.9 Code Conversion (Python - Java) 37
5.10 Code Conversion (Python - Java) 37

List of Tables

2.1 Comparison of Reviewed Literature 11

3.1 Work Breakdown and Responsibilities 20

Chapter 1

Introduction

1.1 Background

Programming is an essential skill required to be proficient in the computing education system and software development. However, it is difficult to understand the programming code written by others, to identify errors in the programs, and to make the programs more efficient. Recently, Artificial Intelligence has greatly improved the tools to assist programmers to write programs and understand the programming code. However, most of the programming tools are developed to perform a specific task. Hence, the programmer needs to use more than one tool to complete a programming task. This is not efficient and disturbs the learning process.RAG-Based Code Assistant project is proposed to develop an intelligent tool to help the programmer understand the programs and make them more efficient.

1.2 Problem Definition

The rapid development of programming languages and the variety of coding styles make it challenging for people to comprehend code that is new to them. This might take a lot of time for people who are new to the subject. Furthermore, activities like developing an algorithm or a flowchart based on a given problem or translating a given code into a different programming language, and optimizing it, make it more difficult.

1.3 Scope and Motivation

The scope of this project is to develop a web-based code assistant that supports small to medium-sized programs written in languages such as Python, Java, and JavaScript.

The assistant's architecture comprises multiple functional modules, like syntax check-

ing, code optimization, code conversion, explanation generation, and the automated creation of algorithms and flowcharts. To improve the quality of its explanations, the system employs a Retrieval-Augmented Generation methodology, which retrieves pertinent contextual information from external knowledge sources, such as programming documentation and open-source repositories.

The project primarily aims at helping students, beginners, and programmers who find complex code difficult to understand. The code assistant can reduce the time users spend analyzing code. Moreover, the code assistant is useful for programmers because it combines several different features.

1.4 Project Objectives

- **Code Understanding and Explanation:** Develop a system that generates clear and structured line-by-line explanations of source code written in Python, Java, and JavaScript using Retrieval-Augmented Generation techniques.
- **Syntax Error Detection:** Implement a syntax checking module that can identify and report syntax errors in programs written in Python, Java, and JavaScript.
- **Code Optimization:** Apply static code optimization techniques such as constant folding, dead code elimination, and copy propagation to improve program efficiency.
- **Code Conversion:** Design a module that automatically converts source code between programming languages such as Python and Java while preserving program functionality.
- **Algorithm and Flowchart Generation:** Create a module that generates algorithms and flowcharts from natural language problem statements.
- **Integrated Platform:** Provide all these functionalities within a single system to simplify program analysis and learning.

1.5 Risks and Challenges

- **Accuracy of AI Models:**

The performance of explanation generation and code conversion modules depends on the quality of language models and training datasets.

• **Retrieval Quality:**

In a RAG-based system, inaccurate or irrelevant information retrieved from the knowledge base may affect the quality of generated responses.

• **Performance Overhead:**

Operations such as embedding generation, retrieval, and API calls may introduce latency during execution.

• **Complex Code Structures:**

Programs with deeply nested logic, unusual syntax, or dynamic features may be difficult to analyze accurately.

• **Dependence on External Resources:**

The system depends on external APIs and knowledge sources, which may sometimes affect availability or performance.

1.6 Assumptions

- The input code given by users is almost structured for analysis.
- External knowledge sources, such as programming documentation and repositories, are available for retrieval operations.
- Large language models used in the system are capable of providing meaningful explanations and suggestions.
- The internet connection required for API access and data retrieval remains stable during system operation.
- Users primarily work with small to medium-sized programs within the supported programming languages.

1.7 Societal/Industrial Relevance

The proposed system has relevance for both educational and industrial environments.

Educational Impact

- Helps students better understand programming concepts through automated explanations.
- Supports learning by generating algorithms and flowcharts from problem statements.
- Reduces the difficulty of interpreting unfamiliar code.

Industrial Impact

- Supports code maintenance by helping programmers quickly understand existing codebases.
- Assists in understanding legacy or unfamiliar codebases, making it easier for teams to maintain and update existing systems. .

By combining artificial intelligence with software engineering tools, the system contributes to more efficient programming practices and enhanced learning experiences.

1.8 Organization of the Report

This report is structured into six main chapters. The chapters are organized as follows:

- **Chapter 1 – Introduction:**
  This chapter provides an overview of the project. It discusses the background, problem definition, scope and motivation, objectives, assumptions, risks, and challenges associated with developing an AI-based code assistant. It also highlights the societal and educational relevance of intelligent programming assistance systems and explains the overall organization of the report.
- **Chapter 2 – Literature Survey:**
  This chapter reviews existing research works related to code understanding, code translation, code optimization, and source code summarization. It discusses various approaches such as Retrieval-Augmented Generation (RAG), transformer-based

models, and Large Language Models (LLMs). The chapter compares different techniques and identifies research gaps that motivate the development of the proposed system.

- **Chapter 3 – System Design:**
  This chapter explains the architecture and design of the proposed AI-powered code assistant. It presents the overall system architecture, design diagrams, and major components of the system. The chapter also describes the algorithms used, software and hardware requirements, module division among team members, expected outputs, and the integration strategy for different modules within the system.

- **Chapter 4 – System Implementation:**
  This chapter describes the implementation details of the proposed system. It includes dataset preparation, syntax checking implementation, code optimization techniques, and the code conversion module. It also explains the implementation of the Retrieval-Augmented Generation (RAG) based explanation module and the algorithm and flowchart generation module.

- **Chapter 5 – Results and Discussion:**
  This chapter presents the results obtained from the implemented system and evaluates its performance. It demonstrates the functionality of different modules such as syntax checking, code optimization, code conversion, explanation generation, and algorithm and flowchart creation. The chapter also discusses the effectiveness of the system in assisting users with programming tasks.

- **Chapter 6 – Conclusion and Future Scope:**
  The final chapter summarizes the key findings and contributions of the project. It highlights how the proposed AI-powered code assistant improves code understanding and analysis. The chapter also outlines possible future enhancements such as supporting additional programming languages, improving model accuracy, expanding the knowledge base, and integrating the system with development environments.

The report concludes with references to relevant research works and appendices that include supporting materials such as presentations, programme outcomes, and CO–PO–PSO mappings.

1.9 Summary of the Chapter

This chapter introduced the concept of an AI-powered RAG-Based Code Assistant designed to support programmers in understanding and improving source code. The background highlighted the growing complexity of programming tasks and the need for intelligent assistance tools. The chapter also described the problem addressed by the project and the motivation for developing an integrated system that combines multiple programming support functions.

Chapter 2

Literature Survey

2.1 Y. Rong, T. Du, R. Li, W. Bao (2025). Integrating LLM-Based Code Optimization with Human-Like Exclusionary Reasoning for Computational Education[1]

This paper introduces a new way to enhance code optimization in large language models by incorporating human-like selective judgment. While LLMs are powerful at generating and improving code, they sometimes make unnecessary or even harmful changes. By adopting this selective approach, the model focuses only on meaningful improvements.

Strengths

- Avoids unnecessary or harmful edits commonly seen in end-to-end fine-tuning approaches.
- Model-agnostic design enables the use of different open-source large language models, thus promoting flexibility

Limitations

- Incorporation of multi-stage decision and filtering mechanisms may result in increased complexity, leading to longer computation times.
- Current focus is primarily on Python, requiring further adaptation to support other programming languages or domains.

Relevance to Project: This framework aids the 'code optimization' feature of the project. The exclusionary reasoning approach helps avoid overwhelming users with unnecessary changes. Applying human-like selective judgment can improve feedback quality and trustworthiness of your assistant.

2.2 Chunyan Zhang, Qinglei Zhou, Meng Qiao(2022).Re_Trans: Combined Retrieval and Transformer Model for Source Code Summarization[2]

The Re Trans model is a model useful for Source Code Summarization (SCS) that utilizes the accuracy of retrieval-based methods with the broad coverage of generative methods. For any code, the model begins by retrieving a summary from semantically close code in its library. A discriminator subsequently decides whether this retrieved summary is optimal. If not, it generates the final summary using a generative Transformer model.

Strengths:

- With its efficient hybrid approach, it can attain state-of-the-art results as well as sophisticated understanding of code structures.

Limitations:

- It may fail if there is no similar code to retrieve, struggles with longer sequences, and demands expensive GPUs for training.

Relevance to Project: The Re Trans model is primarily useful in source code summarization (SCS), which aligns with the assistant's "code explanation" feature. The model's hybrid architecture and its technique of combining sequential and structural (AST) information provide a proven engine for the deep code understanding required for all of the assistant's features.

2.3 Yin, X., Ni, C., Nguyen, T. N., Wang, S., & Yang, X. (2024). Rectifier: Code Translation with Corrector via LLMs[3]

This research proposes a framework called Rectifier, which improves the accuracy of code translation by introducing a two-stage approach. In the first stage, a Large Language Model (LLM) generates the translated code. In the second stage, a lightweight correction model is used to fix syntax and compilation errors in the generated output. This correction model is trained on pairs of invalid and valid code, allowing it to learn common error patterns and improve the reliability of translated programs.

Strengths:

- Improves translation quality by introducing a dedicated correction stage after initial code generation.
- The correction model is lightweight and can be integrated with different LLMs without modifying the base translation model.
- Effectively increases compilation success rate by fixing common syntax and structural errors.

Limitations:

- Requires a dataset of invalid–valid code pairs, which can be difficult to collect and train on.
- Mainly focuses on syntactic correction and may not handle deeper logical or semantic errors.
- Adds additional complexity due to the two-stage architecture.

Relevance to Project: This paper is relevant to the project as it highlights the importance of using a post-processing step to improve the quality of code translation. The idea of separating translation and correction into two stages influenced the design of our code conversion module.

However, instead of training a separate correction model, our project uses a Large Language Model (LLM) as a refinement step after generating the initial translation using a fine-tuned CodeT5-small model. This approach avoids the need for additional training data while still improving syntax correctness and readability. Thus, the paper provides conceptual guidance for improving translation reliability, while our implementation adopts a more practical and lightweight refinement-based approach.

2.4 R. Zhang, H. Li, Z. Liu, and J. Yang.(2024).READSUM: Retrieval-Augmented Adaptive Transformer for Source Code Summarization[4]

READSUM is a model designed to generate high-quality code explanations by combining newly written summaries with information from retrieved examples. It enhances context

understanding by identifying and using similar code and its existing summaries from a database. To achieve a rich representation of the code, READSUM uses both its structural form (AST) and its sequential token flow, integrating these two views with a multi-head self-attention mechanism. This allows it to produce thorough and context-aware explanations that reflect both the code's structure and its sequential nature.

Strengths:

- Enhances code summarization accuracy by combining abstractive generation with retrieved context.
- Improves keyword relevance through the inclusion of summaries from similar code.
- Effectively fuses original and retrieved information for better context understanding.

Limitations

- Relies heavily on the quality and size of the code retrieval database.
- Model complexity is higher, making training and deployment more resource-intensive.

Relevance to Project: The READSUM provides a direct implementation of the Retrieval-Augmented Generation (RAG) framework for source code summarization. READSUM serves as a key example of how to leverage retrieval to improve the contextual accuracy of code explanations, directly aligning with project's goal of generating reliable and context-aware outputs.

2.5 Comparison and Gaps Identified

Paper	**Methodology**	**Strengths**	**Limitations**
Integrating LLM-Based Code Optimization with Human-Like Exclusionary Reasoning for Computational Education	Uses a binary classifier to first decide if a code snippet needs optimization. If yes, it triggers an LLM-based rewriting pipeline (e.g., CodeT5 or GPT-3.5) with prompts, followed by static analysis for validation.	Optimizes only when needed, Maintains semantic correctness, Explains changes clearly, Reduces unnecessary processing.	Tested mainly on Python, Relies heavily on classifier accuracy, May produce simplified or suboptimal suggestions, No dynamic/runtime error validation.
Re_Trans: Combined Retrieval and Transformer Model for Source Code Summarization	Retrieval + Transformer + Discriminator. The model retrieves semantically similar code snippets using Siamese BERT embeddings and feeds both the original and retrieved code into a GCN + BiGRU encoder with a Transformer decoder to generate summaries.	It significantly improves code summarization accuracy by incorporating external context through retrieval.	The model's effectiveness depends on the availability and quality of similar code snippets in the retrieval corpus.
Rectifier: Code Translation with Corrector via LLMs	Uses a micro and universal corrector model (CodeT5+ 220M) fine-tuned on (Invalid Code, Valid Code) pairs to learn common error patterns. Operates as a post-translation repair mechanism.	Cost-efficient. Demonstrated high robustness in repairing errors generated by "unknown" LLMs. Effective at fixing common compilation errors	Struggles with certain model-specific errors.
READSUM: Retrieval-Augmented Adaptive Transformer for Source Code Summarization	Combines retrieval of similar code summaries with abstractive generation, integrating structural (AST) and sequential token features through multi-head self-attention to produce context-rich and keyword-relevant code summaries.	Effectively improves code summarization accuracy by fusing retrieved contextual information with structural and sequential code features, resulting in more precise and context-aware summaries.	Performance depends heavily on the quality of the retrieved code and requires significant computational resources for retrieval and fusion processes.

Table 2.1: Comparison of Reviewed Literature

2.6 Research Gaps Identified

From the reviewed literature, the following gaps have been identified:

- Most current tools focus on individual functions like explanation or conversion, lacking an integrated platform with comprehensive code analysis features.
- Many code assistants offer generic explanations without using retrieval-augmented generation to draw from real developer discussions and documentation.
- There is a noticeable gap in tools that effectively combine deep analysis, such as complexity estimation, with clear visual aids like flowcharts.
- Few existing solutions provide strong cross-language support for code translation, explanation, and optimization within one unified user interface.
- Static explanations are common, but most tools do not support dynamic, line-by-line clarifications or variable role analysis that adjust based on user input.

2.7 Summary of the Chapter

The literature review confirms the power of ASTs, LLMs, and RAG for code analysis. However, it also reveals a lack of unified tools that integrate deep, contextual explanations with code conversion, optimization, and visualization. This gap justifies our project's aim: to build a single, comprehensive AI assistant.

Chapter 3

System Design

3.1 Overall System Architecture

The AI-Powered Code Assistant follows a three-tier architecture consisting of the Client Tier, Backend Tier, and Data & AI Tier. This layered architecture separates the user interface, application logic, and data processing components, making the system easier to maintain, scalable, and efficient.

Client Tier

The Client Tier represents the user interface through which users interact with the system. The interface is implemented using HTML, CSS, and JavaScript and allows users to input source code or problem statements. Users can select different functionalities such as syntax checking, code explanation, optimization, code conversion, algorithm generation, and flowchart creation.

When a user submits code or a prompt, the request is sent to the Flask backend server through HTTP requests. The client interface displays the generated results such as explanations, optimized code, converted code, or flowcharts returned from the backend.

Backend Tier

The Backend Tier is responsible for handling the logic of the application, as well as processing and coordinating different modules in the system. This tier is implemented using Python with Flask as a framework, which serves as an API layer connecting the frontend with the AI components.

When a request is sent, it is processed in different modules

1. Code Analysis and Parsing

The input code is first analyzed with the help of parsing techniques and Abstract Syntax Tree (AST) analysis are used to understand the structure of the program, including variables, loops, functions, and conditions. This stage also helps detect syntax errors and prepares the code for further processing.

2. Modular Processing Services

The backend contains several modules responsible for specific tasks:

- **Syntax Checker Module:**

  Validates the syntax of the input program.

- **Explanation Module:**

  Generates **line-by-line explanations** of code using a **Retrieval-Augmented Generation (RAG)** pipeline.

- **Optimization Module:**

  Performs static analysis and applies optimization techniques such as **constant folding, constant propagation, dead code elimination, unreachable code removal, and common subexpression elimination**.

- **Code Conversion Module:**

  Translates code between **Python and Java** using the **CodeT5 model**, with further refinement performed by a large language model.

- **Algorithm and Flowchart Generation Module:**

  Converts natural language problem statements into algorithms and automatically generates flowcharts using **Graphviz**.

Each module processes the input independently and returns results to the backend, which then forwards the final output to the client interface.

Data & AI Tier

The Data & AI Tier provides the **intelligence and external knowledge required for advanced code analysis and explanation**.

1. Knowledge Sources

The system retrieves contextual programming knowledge from sources such as **GitHub repositories and technical programming articles**. These sources provide examples and explanations that help improve the accuracy of generated responses.

2. Vector Database

The retrieved information is stored as embeddings in a **ChromaDB vector database**. When a user requests a code explanation, the system performs **semantic similarity search** in this database to retrieve the most relevant contextual information.

3. LLM Integration

The retrieved context and the user's code are combined into an **enriched prompt** and sent to a **Large Language Model (LLaMA-3.3 through the Groq API)**. The model generates outputs such as **detailed explanations or algorithm steps**. These results are returned to the backend and finally displayed on the user interface.

3.2 Design Diagrams

- Architecture Diagram – This diagram shows the overall flow of the system starting from user input in the web editor, followed by syntax validation, and then routing through the processing pipeline to modules like explanation (RAG + ChromaDB), code conversion (CodeT5 + LLM), optimization (AST-based), and algorithm/flowchart generation (LLM + Graphviz). The outputs are combined and displayed to the user.

- Sequence Diagram – This diagram shows the step-by-step interaction between the user, frontend, and backend modules. After syntax validation, the request is processed based on the selected task (explanation, conversion, optimization, or flowchart), and the final result is returned to the user interface.

3.3 Component Design

The system follows a modular architecture where multiple components work together to provide an intelligent code analysis and assistance platform.

- **User Interface:** The frontend of the application provides an interface where users can input source code or problem statements and select operations such as syntax checking, code explanation, optimization, conversion, algorithm generation, and flowchart generation. The interface is implemented using HTML, CSS, and JavaScript.

- **Flask API Backend:** The backend is implemented using the Flask web framework in Python. It acts as an API layer that receives requests from the user interface, processes them through different modules, and returns the generated results.

- **Code Analysis and Parsing:** This component performs initial analysis of the input program. It uses parsing techniques and Abstract Syntax Tree (AST) analysis to validate syntax, detect errors, and understand the structure of the code such as variables, loops, conditions, and functions.

- **Explanation Module:** This module generates line-by-line explanations of the source code using a Retrieval-Augmented Generation (RAG) pipeline. The system retrieves relevant information from external sources such as GitHub repositories and GeeksforGeeks articles to provide context-aware explanations.

- **Code Conversion Module:** This module performs automated translation between Python and Java. It uses the CodeT5-small model to generate the initial translated code and then refines the output using a Large Language Model.

- **Optimization Module:** This component performs static code analysis and applies optimization techniques such as constant folding, constant propagation, dead code elimination, unreachable code removal, and common subexpression elimination.

- **Algorithm and Flowchart Generation Module:** This module converts natural language problem statements into algorithms and generates flowcharts. The algo-

rithm steps are extracted and classified, and Graphviz is used to generate visual flowchart diagrams.

- **Vector Database:** The system uses ChromaDB as a vector database to store and retrieve embeddings generated from code chunks and external knowledge sources.
- **LLM Integration:** Large Language Models such as LLaMA 3.3 are accessed through the Groq API to generate explanations, algorithm steps, and improved code outputs.

3.4 Algorithms Used and Design

- **AST Parsing and Tree Traversal:** Used to build and analyze the Abstract Syntax Tree of the input code. This helps understand the program structure and enables further analysis and optimization.
- **Semantic Vector Search (k-NN):** Used in the Retrieval-Augmented Generation pipeline. Embeddings generated from code chunks are stored in ChromaDB and similarity search retrieves the most relevant context.
- **Static Code Optimization Algorithms:** Includes techniques such as constant folding, constant propagation, dead code elimination, unreachable code removal, and common subexpression elimination to improve code efficiency.
- **Regex Parsing Algorithm:** Regular expressions are used to extract algorithm steps from LLM-generated outputs for further processing.
- **Rule-Based Classification Algorithm:** Classifies extracted algorithm steps into flowchart categories such as input, process, decision, output, and terminator.
- **Graph Construction and Layout Algorithms:** Used by Graphviz to automatically arrange nodes and edges to generate readable flowcharts.

3.5 Software and Hardware Requirements

Hardware

Laptop/Desktop with minimum 8 GB RAM, Intel i5 or equivalent processor, 10–20 GB free storage space, and stable internet connectivity.

Software

Operating System: Windows/Linux/macOS
Programming Language: Python 3.x
Web Framework: Flask
Frontend Technologies: HTML, CSS, JavaScript
Database: ChromaDB

Tools

Code Editor: Visual Studio Code
Libraries: Sentence Transformers, Transformers, Torch, AST, Javalang
Flowchart Generation: Graphviz
LLM Access: Groq API (LLaMA 3.3)

3.6 Budget

The project is designed to be cost-effective by primarily using open-source tools and publicly available technologies. Most components such as Python libraries, Flask, ChromaDB, and Graphviz are freely available. Cloud services like Groq API are used for accessing Large Language Models. Therefore, the project does not require a significant external budget.

3.7 Module Division and Work Responsibilities

To ensure structured and parallel development, the project was divided into several modules based on the core components of the AI-Powered Code Assistant. Each team member was assigned specific modules according to their technical strengths and interests. Some

modules required collaboration between members to ensure proper integration and system functionality.

Module Overview

The major modules identified in the system are as follows:

1. **Syntax Checker Module** – Validates the syntax of the input code and detects common programming errors using parsing and static analysis.
2. **Optimization Module** – Performs code analysis and applies compiler-based optimization techniques such as constant folding, constant propagation, and dead code elimination.
3. **Code Conversion Module** – Translates code between programming languages (Python and Java) using a machine learning based approach.
4. **Explanation Module** – Uses Retrieval-Augmented Generation (RAG) to generate detailed line-by-line explanations of code by retrieving relevant knowledge from external sources.
5. **Algorithm and Flowchart Generation Module** – Converts natural language problem statements into algorithm steps and automatically generates corresponding flowcharts.

Work Breakdown and Responsibilities

The detailed work division among the team members is presented in Table 3.1.

Table 3.1: Work Breakdown and Responsibilities

Member(s)	**Module**	**Key Responsibilities**
Benita Benny	Syntax Checker & Optimization Module	Implemented syntax validation for Python, Java, and JavaScript; detected syntax errors and common programming issues; developed static optimization techniques including constant folding, constant propagation, and dead code elimination.
Anithra Syindo	Code Conversion Module	Implemented the code translation system using CodeT5-small for converting programs between Python and Java; integrated LLM-based refinement to improve the correctness and readability of generated code.
Christine Michael	Explanation Module	Developed the Retrieval-Augmented Generation (RAG) pipeline; generated embeddings using Sentence Transformers; implemented semantic retrieval using ChromaDB; integrated LLaMA models through the Groq API to produce detailed line-by-line code explanations.
Aleena Althaf	Algorithm and Flowchart Generation Module	Implemented algorithm generation from natural language prompts using LLMs; extracted structured steps from generated algorithms; classified steps and generated visual flowcharts using Graphviz.

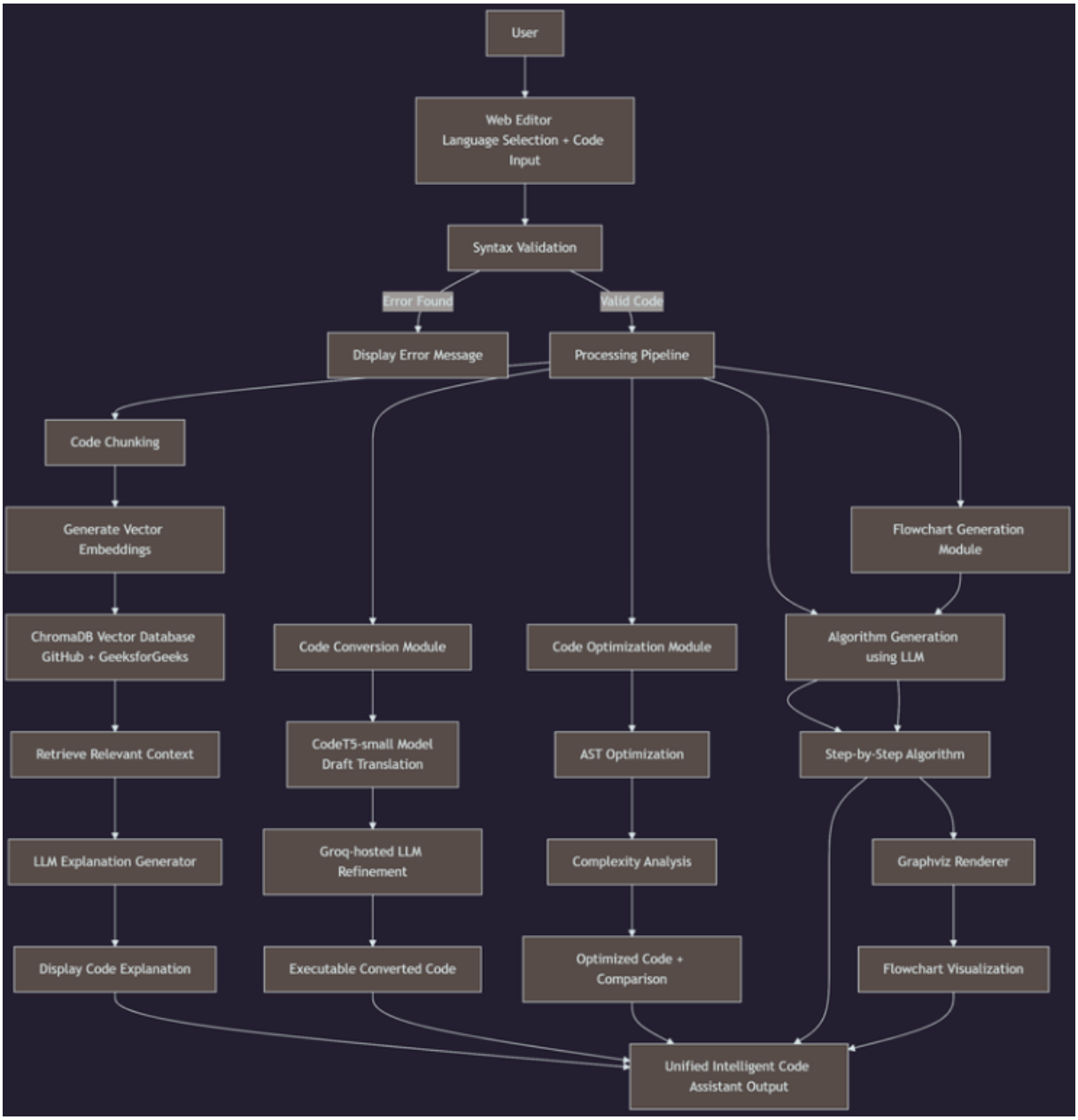


Figure 3.1: Architecture Diagram

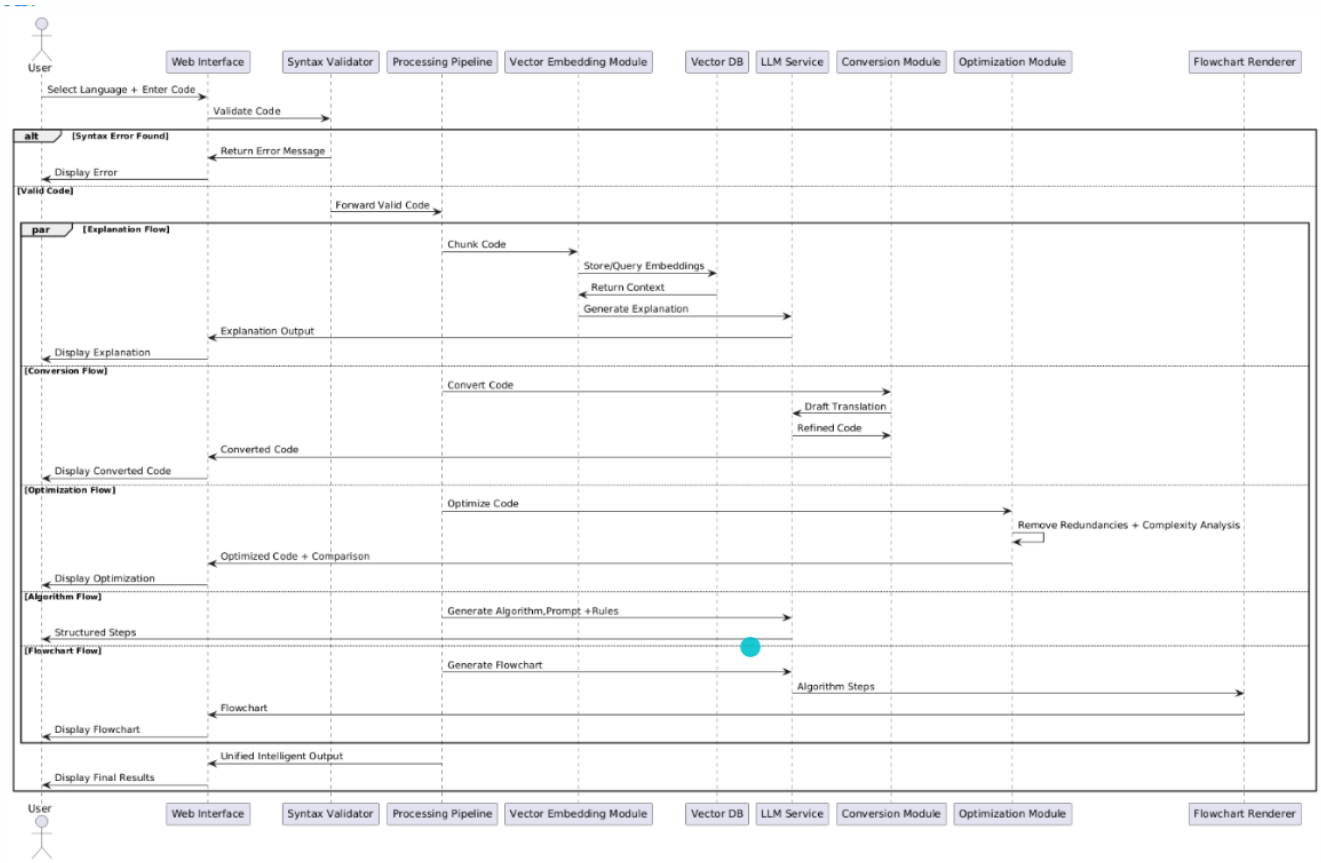


Figure 3.2: Sequence Diagram

Integration Plan

After the individual modules were implemented, all components were integrated into a unified system. The user input is first processed through the Syntax Checker module to validate the code. Once validated, the code can be passed to other modules such as the Optimization, Code Conversion, or Explanation modules, depending on the user's selected operation. Integration testing ensures smooth communication between all modules and correct data flow within the system.

3.8 Key Deliverables - Expected Outputs

- Detect syntax errors in user-provided code (Python, Java, JavaScript)
- Generate optimized versions of the code using static optimization techniques
- Convert code between Python and Java programming languages
- Generate line-by-line explanations of code
- Generate algorithms from natural language problem statements
- Automatically generate flowcharts representing program logic

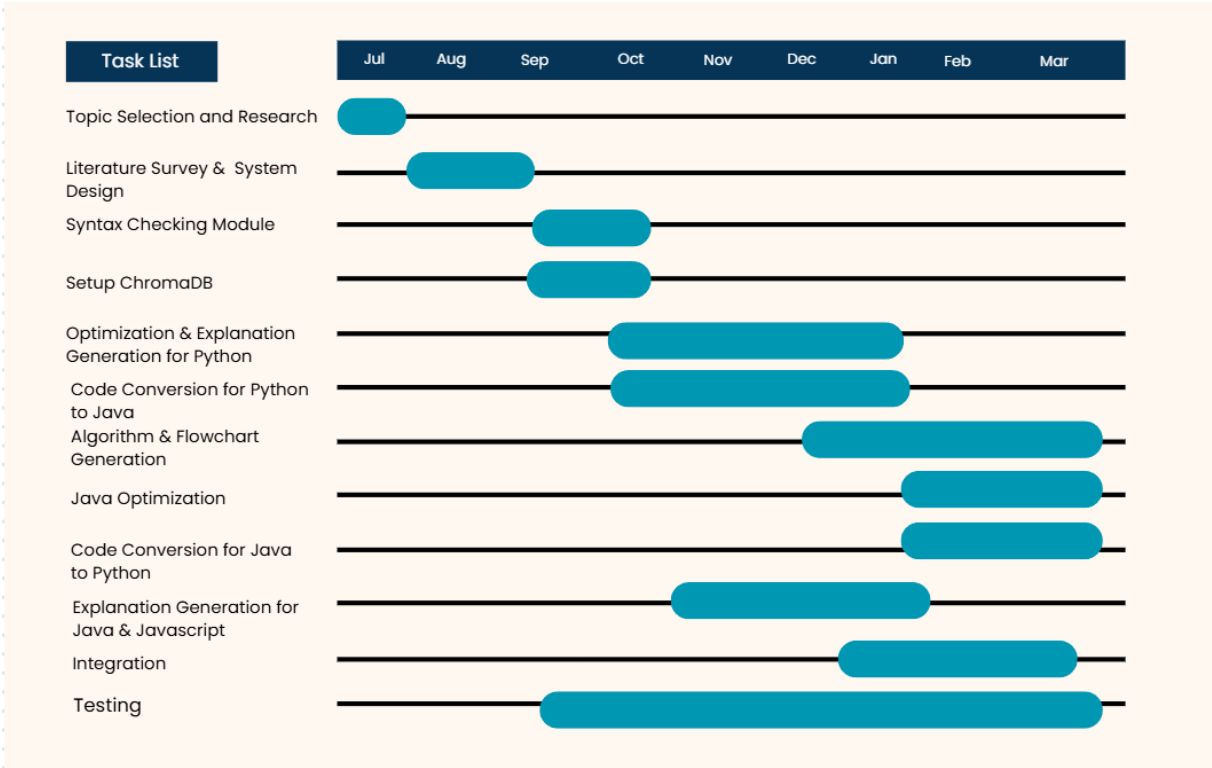


Figure 3.3: Gantt Chart

Chapter 4

System Implementation

4.1 Dataset Preparation and Processing

The dataset used in this project focuses primarily on training and evaluating the code translation module. The dataset contains pairs of equivalent programs written in different programming languages.

Data Collection:

The dataset consists of approximately **2100+ Python–Java and Java–Python program pairs**. Each pair includes a source program and its corresponding translated implementation in another language. The dataset includes various programming constructs such as conditional statements, loops, functions, recursion, and simple data structures. These samples were collected from programming repositories, coding platforms, and open-source educational resources.

Dataset Structure:

Each dataset entry contains:

- Source code written in one programming language
- Equivalent implementation in another programming language
- Program logic preserved across both versions

Dataset Splitting:

To train and evaluate the model effectively, the dataset was divided into two parts:

- **Training set:** 90% of the data used for model learning
- **Validation/Test set:** 10% of the data used for performance evaluation

4.2 Syntax Checker Implementation

The syntax checker module is responsible for identifying errors in source code written in Python, Java, and JavaScript. This module performs static analysis on the input program to detect syntactic and structural issues.

When the user provides source code as input, the system first identifies the programming language selected by the user. Based on the language, a corresponding parser is used to analyze the code structure.

For Python programs, the system uses Python's built-in parsing libraries to check syntax correctness and detect errors. For Java programs, the Javalang parser is used to analyze the code structure and validate syntax rules. JavaScript syntax validation is performed using language-specific parsing methods.

In addition to basic syntax validation, the module also performs several static checks, including:

- Detection of undefined variables
- Identification of unreachable code
- Detection of possible runtime errors such as division by zero

The results of the analysis are returned in a structured **JSON format**, indicating whether the program is syntactically correct and highlighting the detected issues. This information helps users quickly identify and correct errors in their programs.

4.3 Code Optimization Implementation

The optimization module improves the efficiency and readability of programs using **compiler-based static optimization techniques**. The module focuses on analyzing the structure of the program and applying transformations that reduce unnecessary computations.

The optimization process begins by converting the input program into an **Abstract Syntax Tree (AST)**. The AST represents the hierarchical structure of the program, where each node corresponds to a specific programming construct such as expressions, statements, or variables.

After constructing the AST, the system generates a **Control Flow Graph (CFG)**. In the CFG representation, nodes represent program statements and edges represent the possible flow of execution between statements. This representation allows the system to analyze program dependencies and execution paths.

Several optimization techniques like, Constant Folding, Constant Propagation, Copy Propagation, Dead Code Elimination, Unreachable Code Removal, Common Sub-expression Elimination are applied during this process. These optimizations help produce more efficient and simplified versions of the input program while maintaining its original functionality.

4.4 Code Conversion Module

The code conversion module enables automatic translation of programs between **Python and Java**. Instead of relying on rule-based conversion, the system uses a deep learning-based approach to improve translation accuracy.

The translation process is performed using the **CodeT5-small model**, a transformer-based architecture designed for code understanding and generation tasks. The model is fine-tuned on the prepared dataset containing Python–Java and Java-Python code pairs.

During execution, the input source code is first fed to the codeT5-small model which generates the initial translation in the target language.

After the initial translation, a **Large Language Model (LLM)** is used to refine the generated code. This refinement stage helps correct minor syntax issues, improve formatting, and enhance the readability of the output program.

The module ensures that the translated program preserves the original logic and behavior of the input code as closely as possible. Although the current system supports translation between Python and Java, the framework can be extended to support additional programming languages in future versions.

4.5 Code Explanation Module (RAG-Based)

The explanation module generates detailed descriptions of the input code using a **Retrieval-Augmented Generation (RAG)** approach. This method combines external knowledge retrieval with language model generation to produce accurate explanations.

When the user submits source code for analysis, the system divides the code into smaller segments using a **line-based chunking strategy**. Each chunk represents a small group of related lines from the program.

These code chunks are then converted into **vector embeddings** using the **all-MiniLM-L6-v2 Sentence Transformer model**. The embeddings capture the semantic meaning of the code and are stored in the **ChromaDB vector database**.

When a user requests an explanation, the system retrieves relevant contextual information from the knowledge base by performing similarity searches on the embeddings. The knowledge base contains programming examples and explanations collected from sources such as GitHub repositories and technical articles.

The retrieved context, along with the original code chunk, is provided as input to the **LLaMA 3.3 language model** through the **Groq API**. The model then generates a clear, line-by-line explanation describing the functionality of the code.

This retrieval-enhanced approach improves explanation quality by grounding the generated output in relevant programming knowledge.

4.6 Algorithm and Flowchart Generation

The algorithm and flowchart generation module converts natural language problem descriptions into structured algorithms and visual flowcharts.

The process begins when a user enters a problem statement in natural language. The system sends this input to the **LLaMA language model** through the Groq API, which generates a step-by-step algorithm describing the solution.

Once the algorithm text is generated, the system processes it to extract individual steps. This extraction is performed using a **regular expression (Regex) parsing algorithm**, which identifies numbered steps and separates them into structured elements.

Next, the system applies a **rule-based classification algorithm** to categorize each step into one of the following types:

- Input
- Process
- Decision

- Output
- Start/End (terminator)

These categorized steps are then organized into a node–edge structure representing the logical flow of the algorithm.

Finally, the **Graphviz visualization library** is used to generate the flowchart diagram. Graphviz automatically arranges nodes and connections using layout algorithms, producing a clear visual representation of the algorithm. The generated flowchart is saved as an image and displayed to the user.

4.7 Summary of the Chapter

This chapter presented the implementation of the proposed RAG-Based Code Assistant system. It described the preparation and preprocessing of the Python–Java code pair dataset used for training the translation model. The chapter also explained the development of the syntax checker for detecting programming errors, the optimization module that applies compiler-based techniques to improve code efficiency, and the code conversion module that translates programs between Python and Java using the CodeT5-small model. In addition, the RAG-based explanation module was discussed, which retrieves relevant knowledge and generates clear code explanations. Finally, the algorithm and flowchart generation module was implemented to convert natural language problem statements into structured algorithms and visual flowcharts.

Chapter 5

Results and Discussion

This chapter presents the results obtained from the implementation of the RAG-Based Code Assistant system and discusses the performance and effectiveness of the proposed solution. The evaluation focuses on the system's ability to analyze source code, detect syntax errors, optimize programs, convert code between programming languages, generate explanations, and produce algorithms and flowcharts from problem statements. The results demonstrate how the integration of retrieval-augmented generation, machine learning models, and program analysis techniques contributes to improved code understanding and development assistance.

5.1 System Implementation Results

The RAG-Based Code Assistant system was successfully implemented as a web-based platform that integrates multiple modules designed to assist users in understanding and improving programming code. The system accepts user input in the form of source code or natural language problem statements through an interactive user interface.

The syntax checker module analyzes the provided code and identifies errors using language-specific parsing techniques. The system provides feedback regarding syntax correctness and highlights issues that need correction. The code optimization module processes the input program using compiler-based techniques such as constant folding, dead code elimination, and propagation methods to generate a more efficient version of the code.

The code conversion module enables translation between Python and Java programs using the CodeT5-small model combined with a language model for refinement. The generated code preserves the logical structure and functionality of the original program.

The explanation module uses a Retrieval-Augmented Generation approach to generate

detailed line-by-line explanations of the input code. The system retrieves relevant contextual information from stored embeddings and produces understandable explanations using a language model.

The algorithm and flowchart generation module converts natural language problem descriptions into structured algorithms and corresponding flowcharts. The generated algorithms are processed and visualized using Graphviz to create clear diagrams representing the logical flow of the solution.

The screenshots presented in the following sections demonstrate the successful execution of these modules and highlight the overall functionality of the system.

Figure 5.1: Static Syntax Checker

Figure 5.2: Code Optimization Feature

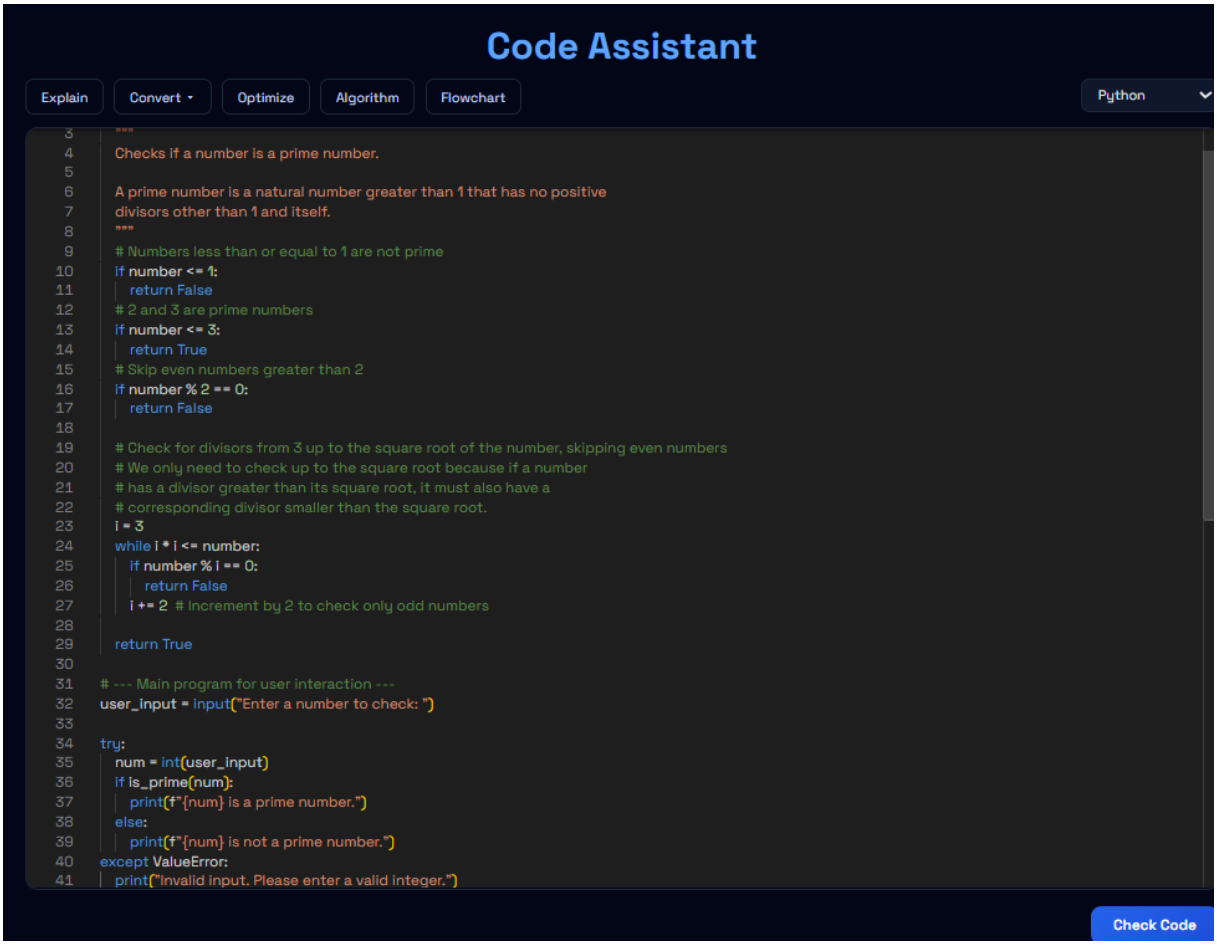


Figure 5.3: Input Code For Explanation

```
Lines 1-41:
Here is a line-by-line explanation of the given Python code:

1. `def is_prime(number):` - This line defines a function named `is_prime` that takes one argument, `number`.

2. `"""Checks if a number is a prime number.` - This line starts a docstring that provides a description of the function.

3. `A prime number is a natural number greater than 1 that has no positive ` - This line continues the docstring, explaining what a prime number
is.

4. `divisors other than 1 and itself."""` - This line ends the docstring.

5. `# Numbers less than or equal to 1 are not prime` - This line is a comment explaining the purpose of the next line of code.

6. `if number <= 1:` - This line checks if the input `number` is less than or equal to 1.

7. `return False` - If the condition in the previous line is true, this line immediately returns `False`, indicating that the number is not prime.

8. `# 2 and 3 are prime numbers` - This line is a comment explaining the purpose of the next line of code.

9. `if number <= 3:` - This line checks if the input `number` is less than or equal to 3.

10. `return True` - If the condition in the previous line is true, this line immediately returns `True`, indicating that the number is prime.

11. `# Skip even numbers greater than 2` - This line is a comment explaining the purpose of the next line of code.

12. `if number % 2 == 0:` - This line checks if the input `number` is even (i.e., if the remainder when divided by 2 is 0).

13. `return False` - If the condition in the previous line is true, this line immediately returns `False`, indicating that the number is not
prime.

14. `# Check for divisors from 3 up to the square root of the number, skipping even numbers` - This line is a comment explaining the purpose of
the next section of code.

15. `# We only need to check up to the square root because if a number ` - This line is a comment explaining why the loop only needs to go up to
the square root of the number.

16. `# has a divisor greater than its square root, it must also have a ` - This line continues the comment from the previous line.

17. `# corresponding divisor smaller than the square root.` - This line ends the comment.

18. `i = 3` - This line initializes a variable `i` to 3, which will be used as a potential divisor.

19. `while i * i <= number:` - This line starts a loop that continues as long as `i` squared is less than or equal to the input `number`.

20. `if number % i == 0:` - This line checks if the input `number` is divisible by `i` (i.e., if the remainder when divided by `i` is 0).

21. `return False` - If the condition in the previous line is true, this line immediately returns `False`, indicating that the number is not
prime.

22. `i += 2 # Increment by 2 to check only odd numbers` - This line increments `i` by 2, effectively skipping even numbers, and includes a comment
explaining why.

23. `return True` - If the loop completes without finding any divisors, this line returns `True`, indicating that the number is prime.

24. `# --- Main program for user interaction ---` - This line is a comment separating the function definition from the main program.

25. `user_input = input("Enter a number to check: ")` - This line prompts the user to enter a number and stores the input in the `user_input`
```

Figure 5.4: Code Explanation Output

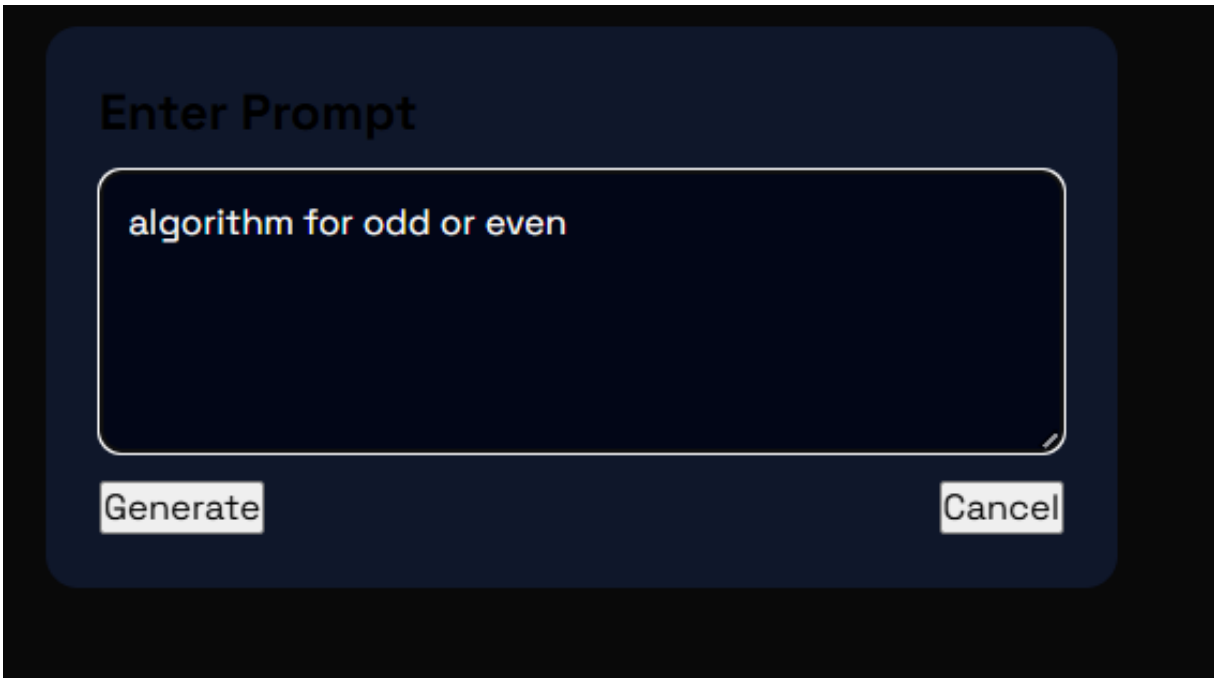


Figure 5.5: Prompt For Algorithm Generation

Figure 5.6: Generated Algorithm

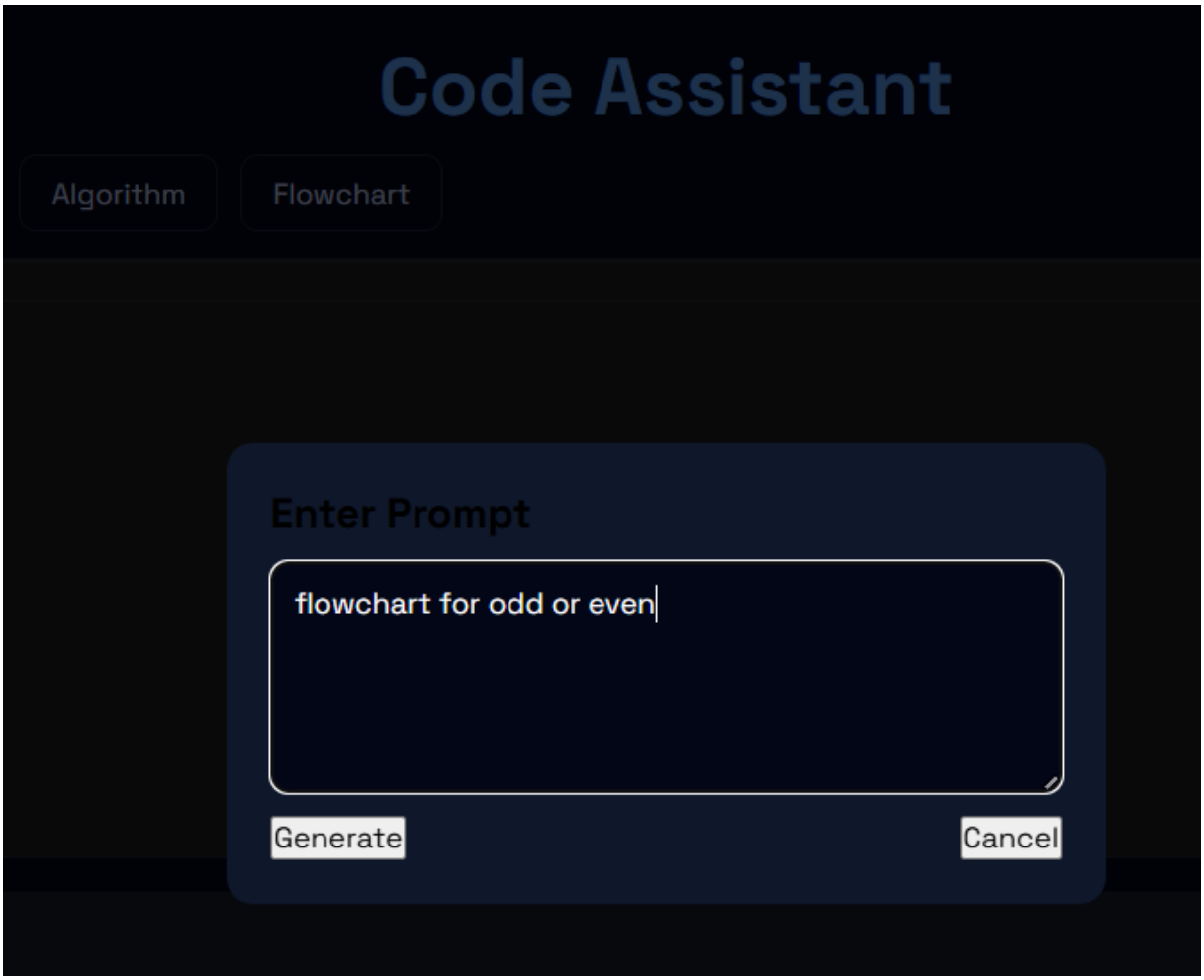


Figure 5.7: Flowchart Prompt

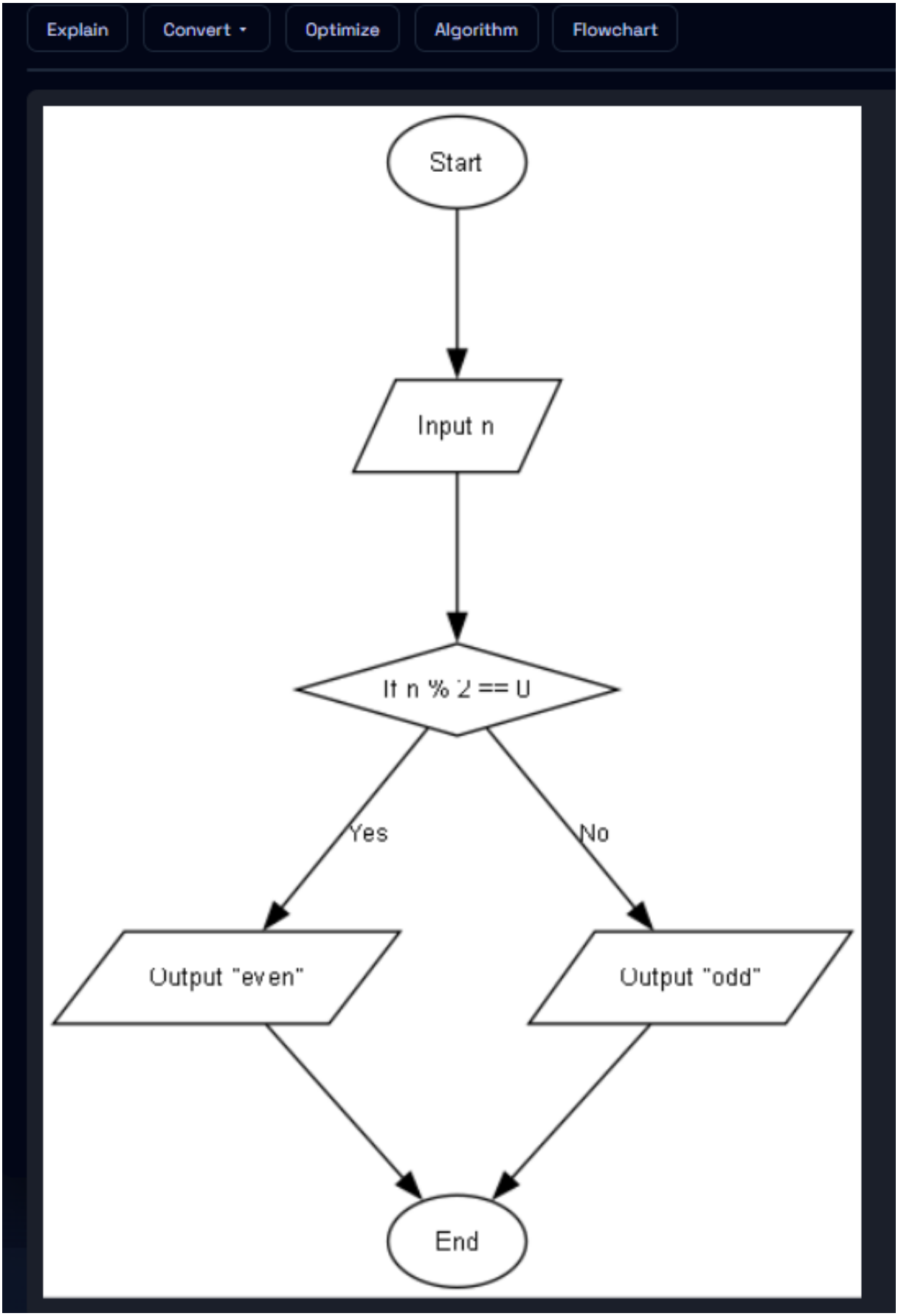


Figure 5.8: Flowchart Generated

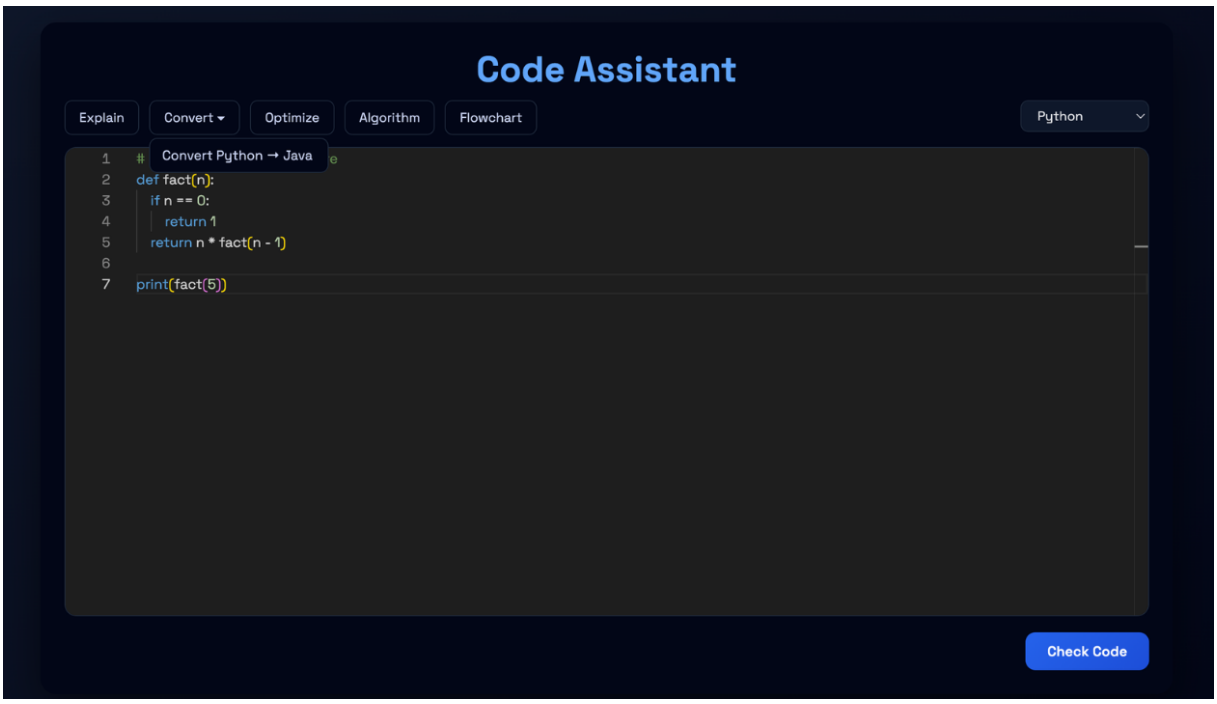


Figure 5.9: Code Conversion (Python - Java)

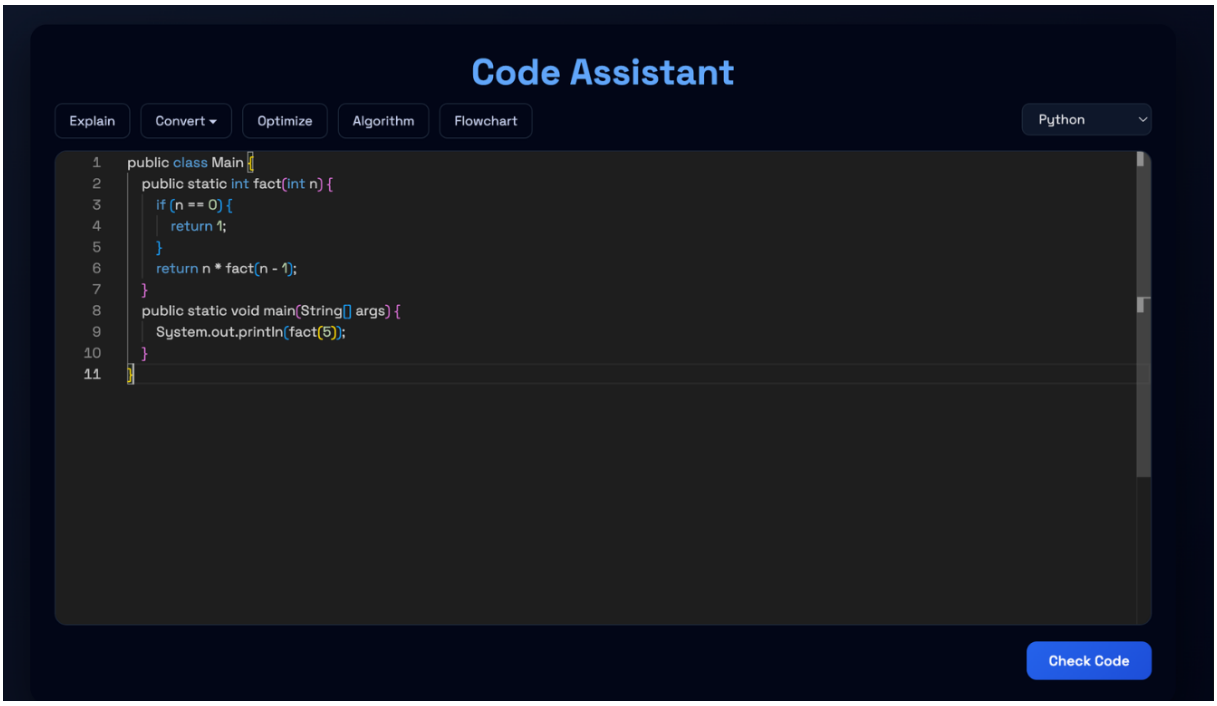


Figure 5.10: Code Conversion (Python - Java)

Chapter 6

Conclusion & Future Scope

The AI-based Code Assistant project addresses the challenges faced by students and beginners when understanding, analyzing, and working with programming code. By integrating multiple AI-driven functionalities into a single platform, the system simplifies several programming-related tasks that are typically handled using separate tools.

The system combines techniques from natural language processing, machine learning, and retrieval-based methods to support different aspects of code analysis. One of the key features of the system is its ability to explain programming code in simple language. To improve the accuracy and relevance of explanations, the project uses a Retrieval-Augmented Generation (RAG) approach. In this method, relevant programming knowledge and contextual information are retrieved from a knowledge base and provided to the language model before generating explanations. This helps produce more reliable and context-aware explanations for the given code.

In addition to code explanation, the system can generate algorithms and flowcharts from prompts, enabling users to visualize the logical structure of programs more clearly. These visual representations help beginners better understand program flow and decision-making processes.

Another important component of the project is the code translation module, which enables conversion between programming languages. The project also includes a code optimization module that performs safe source-level optimizations to improve code efficiency and structure without altering the original program behavior. By integrating these capabilities—code explanation with RAG, algorithm and flowchart generation, code translation, and optimization—into a unified system, the project provides a practical tool that supports programming education and experimentation.

6.1 Future Scope

Although the current system successfully integrates multiple AI-based functionalities such as code explanation, algorithm and flowchart generation, code conversion, and code optimization within a single platform, there are several areas where the system can be further improved and expanded.

One possible improvement is expanding support for additional programming languages. Currently, the system focuses on a limited set of languages, but future versions could support more languages such as C, C++, and other widely used programming languages to make the system more versatile.

Another potential enhancement is improving the accuracy and capability of the code conversion module by using larger datasets and more advanced models. This would allow the system to handle more complex programs and better preserve program logic during translation.

The system could also be integrated with development environments such as IDEs or code editors, allowing programmers to access explanation, conversion, and optimization features directly while writing code. This would make the tool more practical for real-world development scenarios.

Additionally, the Retrieval-Augmented Generation (RAG) mechanism used for code explanations could be expanded with a larger and more comprehensive knowledge base. This would help generate more detailed and context-aware explanations for different programming concepts.

Future work could also focus on improving the user interface and visualization features, enabling clearer algorithm representations, interactive flowcharts, and better user interaction with the system.

Overall, with further improvements in model capabilities, dataset expansion, and system integration, the AI-based Code Assistant has the potential to become a more powerful tool for programming education and code analysis.

References

[1] Y. Rong, T. Du, R. Li, and W. Bao, "Integrating LLM-based code optimization with human-like exclusionary reasoning for computational education," *Journal of King Saud University - Computer and Information Sciences*, 2025.

[2] Y. Chen, X. Liu, F. Wu, and X. Li, "Re_trans: Combined retrieval and transformer model for source code summarization," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, San Francisco, USA, Dec. 2023, pp. 1456–1467.

[3] X. Yin, C. Ni, T. N. Nguyen, S. Wang, and X. Yang, "Rectifier: Code translation with corrector via LLMs," vol. 1, no. 1, Jul. 2024.

[4] R. Zhang, H. Li, Z. Liu, and J. Yang, "Readsum: Retrieval-augmented adaptive transformer for source code summarization," *Information and Software Technology*, vol. 152, p. 106946, 2024.

[5] C. Zhang, Q. Zhou, M. Qiao, K. Tang, L. Xu, and F. Liu, "Re Trans: Combined Retrieval and Transformer Model for Source Code Summarization," Entropy, vol. 24,no. 10, p. 1372, 2022.

Appendix A: Presentation

RAG BASED CODE ASSISTANT

100% Project Presentation

Project Guide : Dr. Anita John
Group 10

Team Members:
Aleena Althaf (U2203217)
Anithra Syindo (U2203044)
Benita Benny (U2203065)
Christine Michael (U2203070)

CONTENTS

1. Problem Definition
2. Project Objective
3. Purpose & Need
4. Literature Survey
5. Proposed Method
6. Architecture Diagram
7. Sequence Diagram
8. Module Description
9. Results & Outputs
10. Assumptions
11. Work Breakdown
12. Hardware Requirements
13. Software Requirements
14. Datasets Used
15. Gantt Chart
16. Risks & Challenges
17. Conclusion
18. References

PROBLEM DEFINITION

- The rapid growth of programming languages and the increasing diversity of coding styles have made understanding unfamiliar code a significant challenge, particularly for students and beginners.

- Interpreting code often requires considerable time and effort, especially for individuals with limited technical experience.

- Additionally, tasks such as generating an algorithm or a flowchart from a prompt, translating code between programming languages, and optimizing it, further add to the difficulty.

PROBLEM OBJECTIVE

To develop an integrated AI-based system that helps users understand, correct, convert, and optimize small to medium-sized programs, while also generating algorithms and flowcharts from prompts to support learning and code analysis. The system supports programming languages such as Python, Java, and JavaScript and is primarily designed for educational purposes.

PURPOSE AND NEED

- The purpose of this project is to make programming code easier to understand and analyze, especially for students. Many existing tools only perform single tasks such as code explanation or debugging, which makes the learning and development process fragmented.

- There is a need for a unified intelligent system that can not only explain code but also detect errors, convert code between programming languages, generate algorithms and flowcharts, and optimizations in one platform.

- Such a system helps users save time, improve code comprehension, and learn programming concepts more effectively.

LITERATURE SURVEY

Paper Title	Method	Advantage	Limitation
Search-Based Large Language Models (SBLLMs)	Iterative search-based code optimization using LLM generated candidates	SBLLMs can discover highly efficient code by exploring multiple optimization possibilities	They require significant computational resources due to iterative generation and evaluation of multiple candidate solutions.
Re_Trans: Combined Retrieval and Transformer Model for Source Code Summarization	Retrieval-Augmented Generation using AST + Transformer	Improves code explanation accuracy using retrieved examples.	Performance depends on the quality of embeddings and vector database.
Code Translation with Corrector via LLMs	Two-stage system: LLM translation + corrector model	Improves compilation success rate of translated code.	Cannot fix deep semantic errors and requires more training and huge dataset.
READSUM: Retrieval-Augmented Adaptive Transformer for Source Code Summarization	Retrieval + adaptive transformer summarization	Generates more informative summaries using similar code examples.	Higher computational cost and system complexity.

LITERATURE SURVEY

Paper Title: Search-Based LLMs for Code Optimization [1]

Authors: Shuzheng Gao, Cuiyun Gao, Wenchao Gu, Michael R. Lyu

Published in: ACM 47th International Conference on Software Engineering

- Search-based LLM optimization treats code optimization as a search problem, generating multiple optimized candidates and selecting the best solution.
- The optimization process is iterative.
- Instead of producing a single result, the system:

1. Generates multiple candidate optimized versions
2. Evaluates their performance
3. Selects the best optimized code

LITERATURE SURVEY

Advantages

- Learns from previous optimized solutions
- Produces high-performance code

Disadvantages

- Requires multiple iterations
- Computationally expensive
- Depends heavily on LLM performance

LITERATURE SURVEY

Paper Title: Re_Trans: Combined Retrieval and Transformer Model for Source Code Summarization [2]

Authors: Chunyan Zhang, Qiang Zhou, Meng Qiao, Ke Tang, Lianqiu Xu, Fudong Liu

Journal: Entropy

The proposed system is the Re_Trans model, which combines semantic code retrieval with transformer-based generation to produce accurate code summaries. Re_Trans incorporates Abstract Syntax Tree (AST) analysis along with Graph Convolutional Networks (GCN) and Bidirectional Gated Recurrent Units (BiGRU) to encode the structural and semantic information of code. A transformer decoder is then used to generate the final summary, and a discriminator selects the most suitable output. This hybrid retrieval–generation approach improves semantic understanding and achieves better performance than standalone transformer-based methods.

LITERATURE SURVEY

Advantages

- Produces more accurate code summaries using Retrieval-Augmented Generation (RAG).
- Combines retrieved code examples with transformer generation for better code understanding.
- Improves explanation quality by using external knowledge.
- Similar architecture to our ChromaDB + LLM system, making it practical for real code explanation tasks.

Disadvantages

- Performance depends on the quality of the vector database.
- Incorrect embeddings may lead to irrelevant code retrieval.

LITERATURE SURVEY

Paper Title: Code Translation with Corrector via LLMs [3]

Authors: X. Yin, C. Ni, T. N. Nguyen, S. Wang, X. Yang

Proposes a two-stage framework for improving code translation accuracy.

The system first generates translated code using a pretrained Large Language Model.

Then, a micro corrector model (CodeT5+ 220M) fine-tuned on (Invalid Code, Valid Code) pairs is used to repair syntactic and compilation errors.

The correction stage acts as a post-translation repair mechanism to improve robustness and compilation success rate.

LITERATURE SURVEY

Advantages

- Improves compilation success rate significantly.
- Cost-efficient due to smaller corrector model.
- Modular architecture (translation + correction separated).
- Effective at fixing common syntax and structural mistakes.

Disadvantages

- May not fix deep semantic or logical errors.
- Requires large dataset of invalid-valid code pairs for training.
- Adds architectural complexity (two-model pipeline).

LITERATURE SURVEY

Paper Title: READSUM: Retrieval-Augmented Adaptive Transformer for Source Code Summarization

Journal: IEEE Access (2023)

Authors: Yunseok Choi, Cheolwon Na, Hyojun Kim, Jee-Hyong Lee

The paper proposes **READSUM**, a retrieval-augmented transformer model designed for automatic source code summarization. It first retrieves similar code snippets and their corresponding summaries to provide contextual guidance for generation. An adaptive transformer encoder then combines structural and sequential representations of the source code using self-attention mechanisms. A fusion module integrates the retrieved summary information with the generated output to enhance informativeness and relevance.

LITERATURE SURVEY

Advantages

- Combines retrieval and generation to improve summary accuracy and contextual relevance.
- Uses similar code examples to produce more informative and meaningful summaries.
- Adaptive transformer captures both structural and semantic features of source code.
- Fusion mechanism improves keyword coverage and reduces missing details.

Disadvantages

- Performance depends heavily on the quality of the retrieval database.
- May retrieve irrelevant examples if similarity matching is weak.
- Requires well-structured and large datasets for effective training.
- Model complexity makes implementation and deployment more challenging.

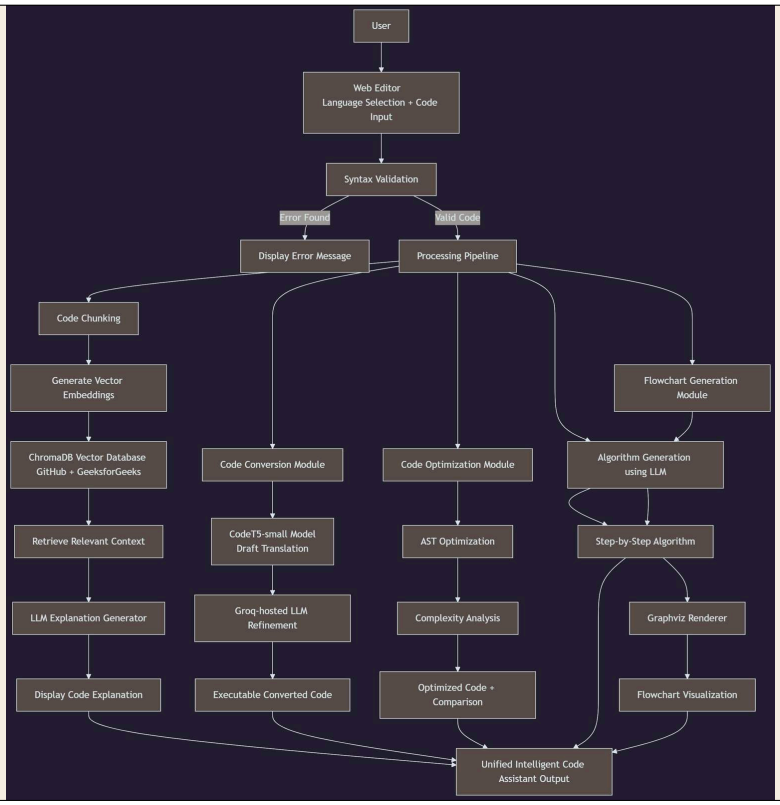
PROPOSED METHOD

- The proposed system is an AI-powered code assistant that combines Retrieval-Augmented Generation (RAG) with Large Language Models (LLMs) to help users understand and improve code.

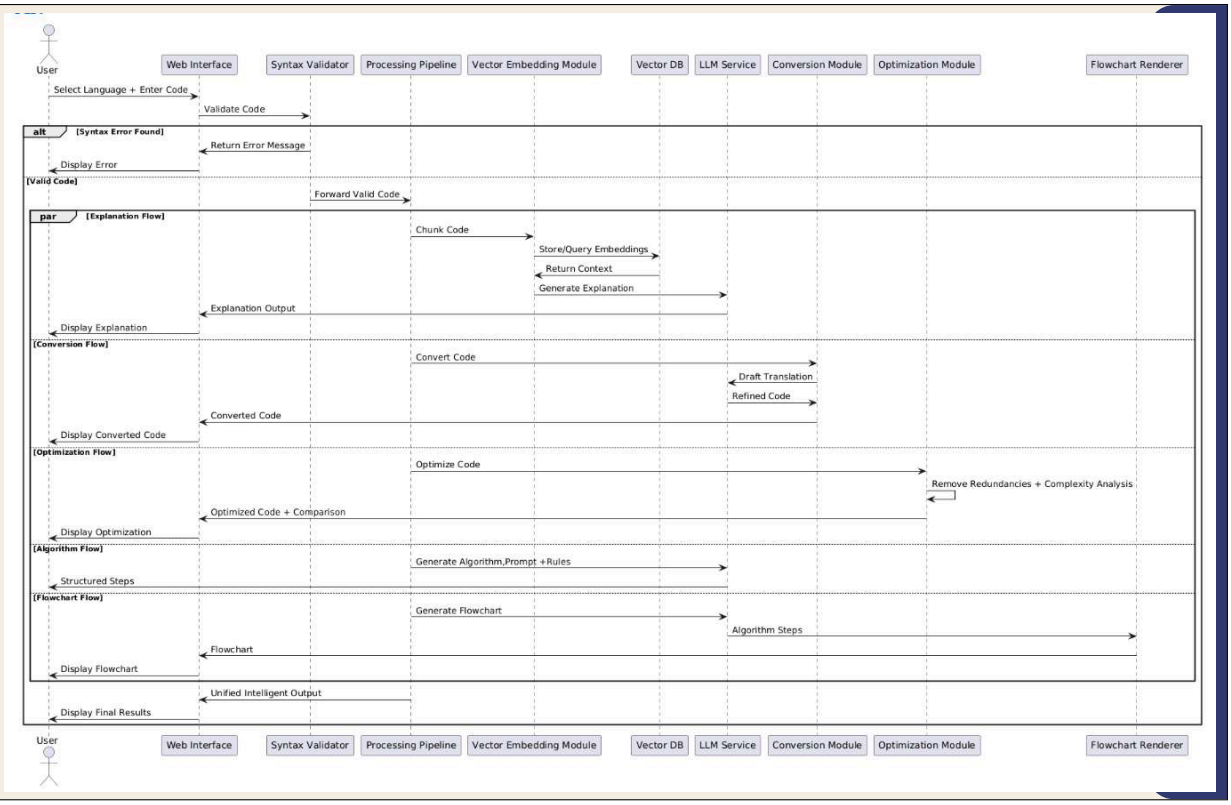
- The system first statically analyzes the input code, retrieves relevant information from external sources such as GitHub and GeeksforGeeks, and then uses this retrieved context to generate accurate explanations and suggestions.

- The method also includes modules for syntax checking, code optimization, code conversion between programming languages, and algorithm & flowchart generation. These components work together within a single platform to make code comprehension easier.
- Languages Supported: Python, Java, JavaScript

ARCHITECTURE DIAGRAM



SEQUENCE DIAGRAM



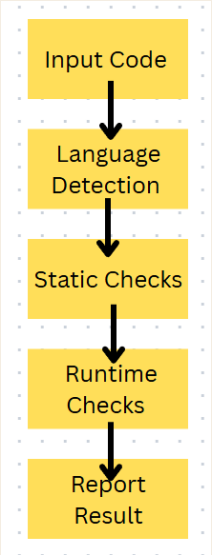
MODULES

Module 1 : Syntax Checker Module
Module 2 : Optimization Module
Module 3 : Code Conversion Module
Module 4 : Explanation Module
Module 5 : Algorithm and Flowchart Generation Module

MODULE 1 - SYNTAX CHECKER

- Accepts source code input and identifies the selected programming language.
- Performs language-specific syntax validation (Python, Java, JavaScript).
- Uses parsing and compilation techniques to detect syntax errors.
- Conducts static analysis checks such as undefined variables, unreachable code, and possible division by zero.
- Performs basic runtime validation to detect execution errors.
- Returns results in structured JSON format including syntax status and detected issues.

Execution Flow



MODULE 2 - OPTIMIZATION

- Compiler-Based Static Optimization
- Implemented for Python and Java
- The optimizer first converts the input Java program into an **Abstract Syntax Tree (AST)**
- This tree structure represents the syntactic hierarchy of the program, enabling accurate analysis and transformation of expressions, variables, and statements.
- The system builds a **Control Flow Graph (CFG)** from the AST where each node represents a program statement and edges represent the flow of execution.
- This representation helps in understanding dependencies and enables more structured program analysis.

MODULE 2 - OPTIMIZATION

Optimization Techniques employed:

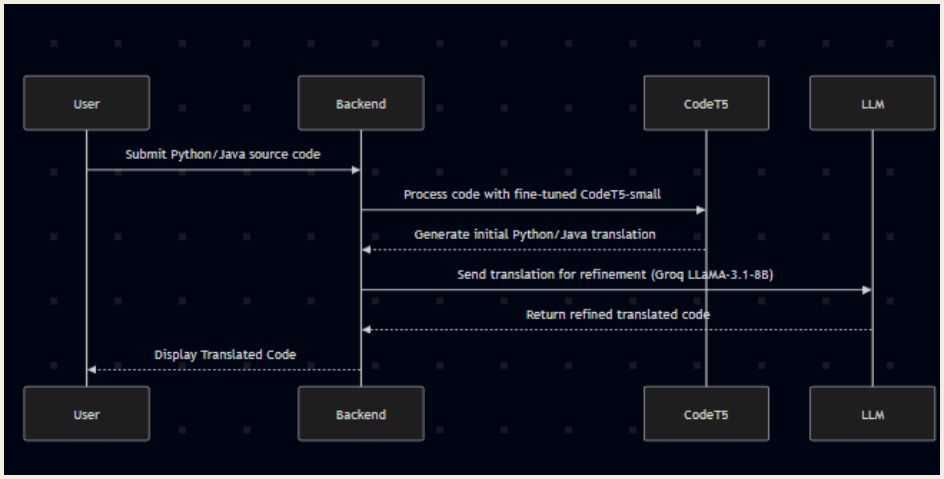
- Constant Folding: Computes constants early.
- Constant Propagation: Reuses known constant values.
- Copy Propagation: Eliminates unnecessary variable copies by replacing aliases.
- Dead Code Elimination: Removes unused code.
- Unreachable Code Removal: Deletes non-executable code paths.
- Common Subexpression Elimination: Avoids repeated computations.

Execution Flow

MODULE 3 - CODE CONVERSION (PYTHON ⇄ JAVA)

- Performs automated translation of Python source code to Java and vice-versa.
- Uses a hybrid deep learning approach instead of rule-based conversion.
- CodeT5-small generates the initial Java/Python translation.
- A Large Language Model (LLM) refines the output to improve accuracy and correctness.
- The LLM is used to correct minor syntax errors and improve the overall quality of the generated Java/Python code beyond the base model output.
- Preserves semantic equivalence and functional behavior of the original program.
- Current implementation supports Python ⇄ Java.
- Scope can be extended to other language pairs in future.

MODULE 3 - CODE CONVERSION



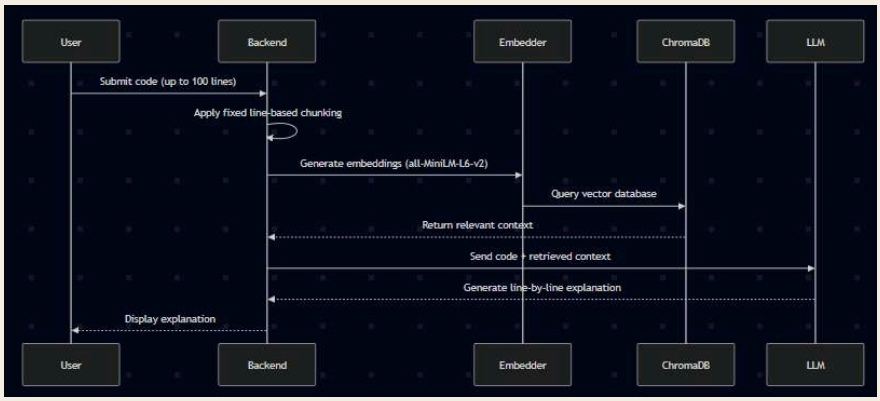
Execution Flow

MODULE 4 - EXPLANATION

- The module generates line-by-line explanations of source code using a Retrieval-Augmented Generation (RAG) approach.
- The system supports Python, Java, and JavaScript programming languages.
- The user provides input code (up to around 100 lines) for analysis.
- The code is divided using fixed line-based chunking, where small group of lines is treated as a chunk.
- Each chunk is converted into vector embeddings using the all-MiniLM-L6-v2 model from Sentence Transformers.
- The embeddings are stored and searched in the ChromaDB vector database.
- The system retrieves relevant contextual information from the knowledge base using RAG.
- The knowledge base includes code and explanation from GitHub repositories and GeeksforGeeks articles.

MODULE 4 - EXPLANATION

- The retrieved context and code chunk are sent to the LLaMA-3.3-70B-Versatile model through the Groq API.
- The model generates clear line-by-line explanations of the input code for the user.



MODULE 5 - ALGORITHM & FLOWCHART GENERATION

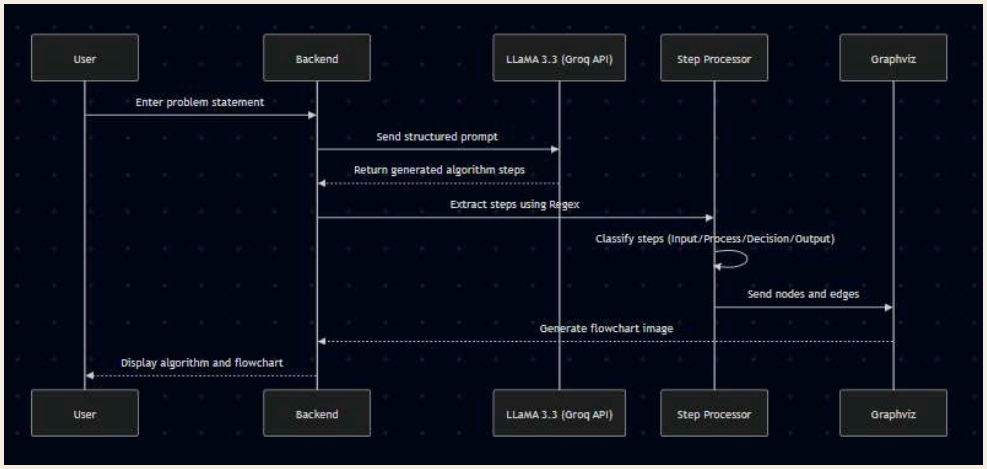
- The module converts a natural language problem statement into an algorithm and flowchart automatically using an LLM.
- The user provides a problem statement in natural language as input to the system.
- The system uses the Groq API with the LLaMA-3.3 model to generate a step-by-step algorithm for the given problem.
- The generated algorithm text is processed to extract individual steps.
- Each step is classified as input, output, decision, process, or terminator for flowchart creation.
- The classified steps are organized into a structured flow of nodes and connections.
- Graphviz is used to convert the structured steps into a visual flowchart diagram.
- The system automatically generates and saves the flowchart image for the user.

MODULE 5 - ALGORITHM & FLOWCHART GENERATION

Algorithms Used:

- Regex Parsing Algorithm : Uses regular expression matching to extract numbered algorithm steps from the LLM output.
- Rule-Based Classification Algorithm : Applies predefined rules to categorize steps into input, process, decision, output, and terminator.
- Graph Construction Algorithm : Builds a node–edge structure where each step becomes a node and logical flow becomes edges.
- Graphviz Layout Algorithm : Graphviz uses layout algorithms to automatically arrange nodes and edges in the flowchart.

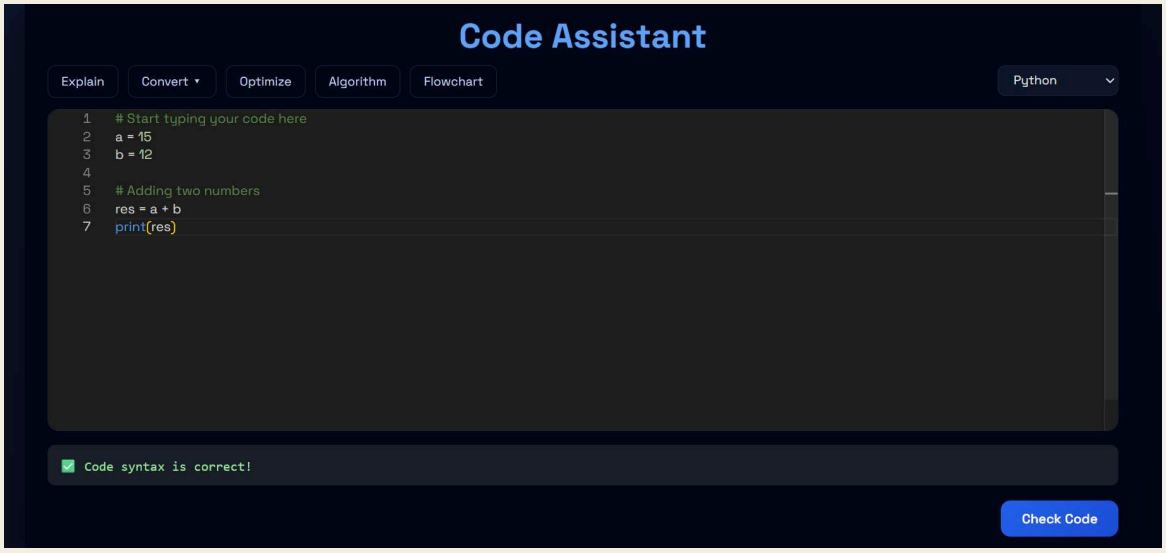
MODULE 5 - ALGORITHM & FLOWCHART GENERATION



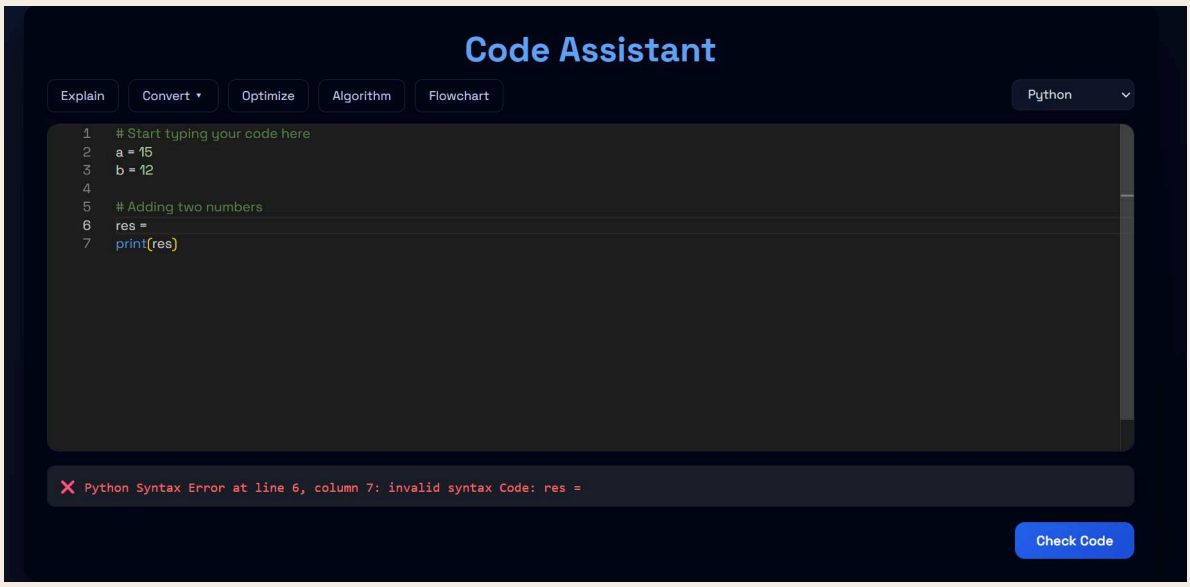
Execution Flow

RESULTS AND OUTPUT SCREENSHOTS

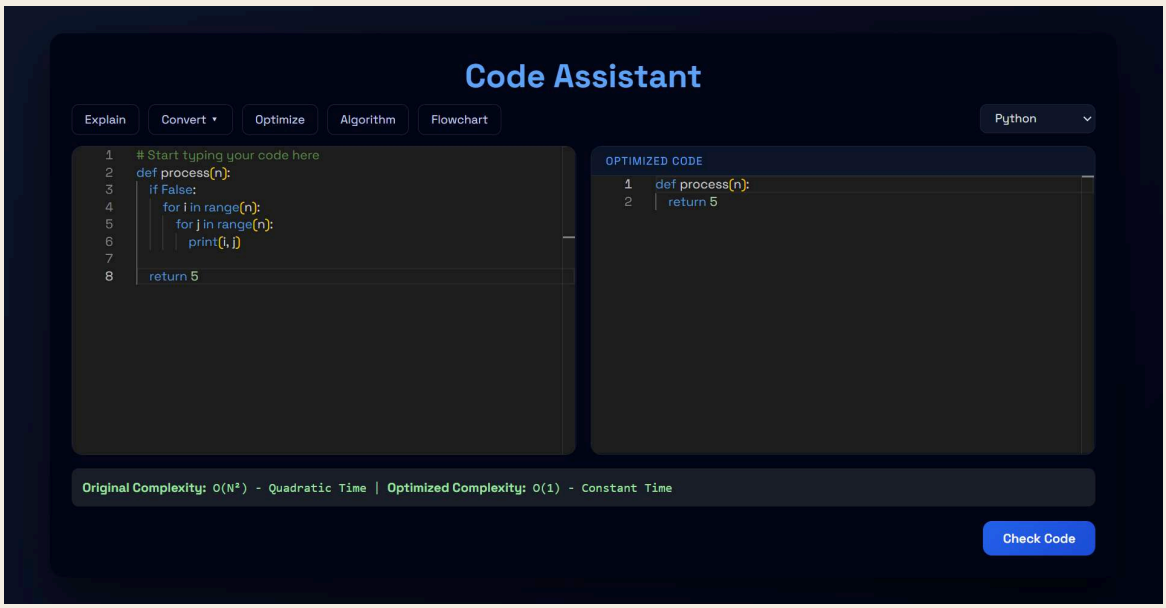
MODULE 1 - SYNTAX CHECKER



MODULE 1 - SYNTAX CHECKER



MODULE 2 - OPTIMIZATION



MODULE 3 - CODE CONVERSION

Python to Java

Code Assistant

Explain | Convert | Optimize | Algorithm | Flowchart | Python

Convert Python → Java

```python
# 
def fact(n):
    if n == 0:
        return 1
    return n * fact(n - 1)

print(fact(5))
```

Check Code

Code Assistant

Explain | Convert | Optimize | Algorithm | Flowchart | Python

```java
public class Main {
    public static int fact(int n) {
        if (n == 0) {
            return 1;
        }
        return n * fact(n - 1);
    }
    public static void main(String[] args) {
        System.out.println(fact(5));
    }
}
```

Check

MODULE 3 - CODE CONVERSION

Java to Python

Code Assistant

Explain | Convert | Optimize | Algorithm | Flowchart | Java

Convert Java → Python

```java
public class Main {
    public static void main(String[] args) {

        int num = 29;
        boolean flag = true;

        if (num <= 1) {
            flag = false;
        } else {
            for (int i = 2; i < num; i++) {
                if (num % i == 0) {
                    flag = false;
                    break;
                }
            }
        }

        if (flag) {
            System.out.println("Prime");
        } else {
            System.out.println("Not Prime");
```

Check Code

Code Assistant

Explain | Convert | Optimize | Algorithm | Flowchart

```python
num = 29
flag = True

if num <= 1:
    flag = False
else:
    for i in range(2, num):
        if num % i == 0:
            flag = False
            break

if flag:
    print("Prime")
else:
    print("Not Prime")
```

MODULE 4 - EXPLANATION

Code Assistant

Explain | Convert | Optimize | Algorithm | Flowchart | Python

```python
3   """
4   Checks if a number is a prime number.
5
6   A prime number is a natural number greater than 1 that has no positive
7   divisors other than 1 and itself.
8   """
9   # Numbers less than or equal to 1 are not prime
10  if number <= 1:
11      return False
12  # 2 and 3 are prime numbers
13  if number <= 3:
14      return True
15  # Skip even numbers greater than 2
16  if number % 2 == 0:
17      return False
18
19  # Check for divisors from 3 up to the square root of the number, skipping even numbers
20  # We only need to check up to the square root because if a number
21  # has a divisor greater than its square root, it must also have a
22  # corresponding divisor smaller than the square root.
23  i = 3
24  while i * i <= number:
25      if number % i == 0:
26          return False
27      i += 2  # Increment by 2 to check only odd numbers
28
29  return True
30
31  # --- Main program for user interaction ---
32  user_input = input("Enter a number to check: ")
33
34  try:
35      num = int(user_input)
36      if is_prime(num):
37          print(f"{num} is a prime number.")
38      else:
39          print(f"{num} is not a prime number.")
40  except ValueError:
41      print("Invalid input. Please enter a valid integer.")
```

Check Code

MODULE 4 - EXPLANATION

Lines 1-41:
Here is a line-by-line explanation of the given Python code:

1. `def is_prime(number):` - This line defines a function named `is_prime` that takes one argument, `number`.

2. `"""Checks if a number is a prime number.` - This line starts a docstring that provides a description of the function.

3. `A prime number is a natural number greater than 1 that has no positive ` - This line continues the docstring, explaining what a prime number is.

4. `divisors other than 1 and itself."""` - This line ends the docstring.

5. `# Numbers less than or equal to 1 are not prime` - This line is a comment explaining the purpose of the next line of code.

6. `if number <= 1:` - This line checks if the input `number` is less than or equal to 1.

7. `return False` - If the condition in the previous line is true, this line immediately returns `False`, indicating that the number is not prime.

8. `# 2 and 3 are prime numbers` - This line is a comment explaining the purpose of the next line of code.

9. `if number <= 3:` - This line checks if the input `number` is less than or equal to 3.

10. `return True` - If the condition in the previous line is true, this line immediately returns `True`, indicating that the number is prime.

11. `# Skip even numbers greater than 2` - This line is a comment explaining the purpose of the next line of code.

12. `if number % 2 == 0:` - This line checks if the input `number` is even (i.e., if the remainder when divided by 2 is 0).

13. `return False` - If the condition in the previous line is true, this line immediately returns `False`, indicating that the number is not prime.

14. `# Check for divisors from 3 up to the square root of the number, skipping even numbers` - This line is a comment explaining the purpose of the next section of code.

15. `# We only need to check up to the square root because if a number ` - This line is a comment explaining why the loop only needs to go up to the square root of the number.

16. `# has a divisor greater than its square root, it must also have a ` - This line continues the comment from the previous line.

17. `# corresponding divisor smaller than the square root.` - This line ends the comment.

18. `i = 3` - This line initializes a variable `i` to 3, which will be used as a potential divisor.

19. `while i * i <= number:` - This line starts a loop that continues as long as `i` squared is less than or equal to the input `number`.

20. `if number % i == 0:` - This line checks if the input `number` is divisible by `i` (i.e., if the remainder when divided by `i` is 0).

21. `return False` - If the condition in the previous line is true, this line immediately returns `False`, indicating that the number is not prime.

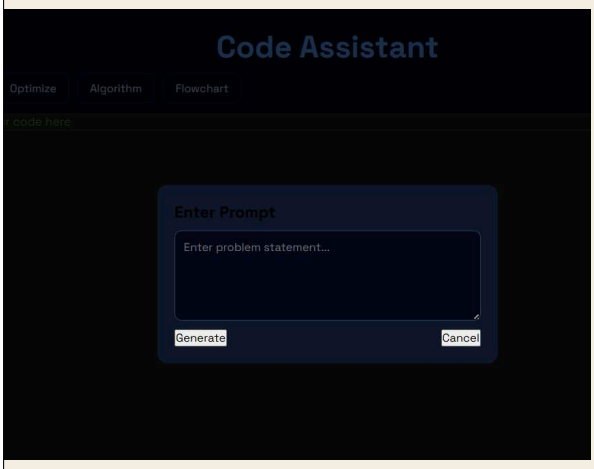
22. `i += 2 # Increment by 2 to check only odd numbers` - This line increments `i` by 2, effectively skipping even numbers, and includes a comment explaining why.

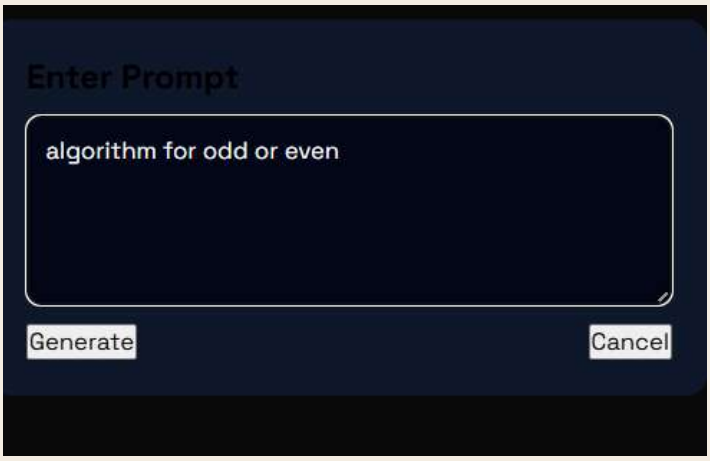
23. `return True` - If the loop completes without finding any divisors, this line returns `True`, indicating that the number is prime.

24. `# --- Main program for user interaction ---` - This line is a comment separating the function definition from the main program.

25. `user_input = input("Enter a number to check: ")` - This line prompts the user to enter a number and stores the input in the `user_input`

MODULE 5 - ALGORITHM & FLOWCHART GENERATION

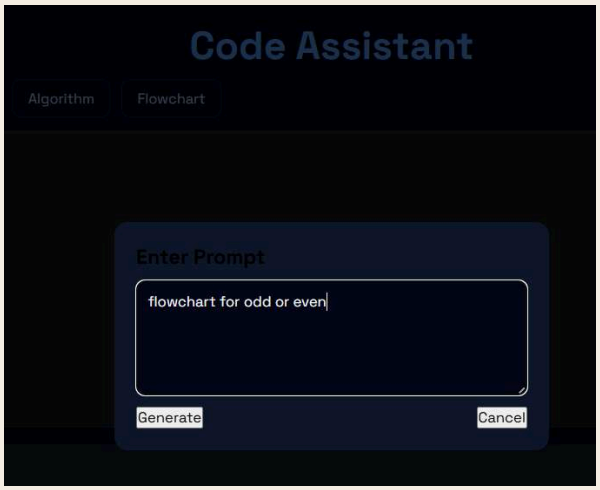


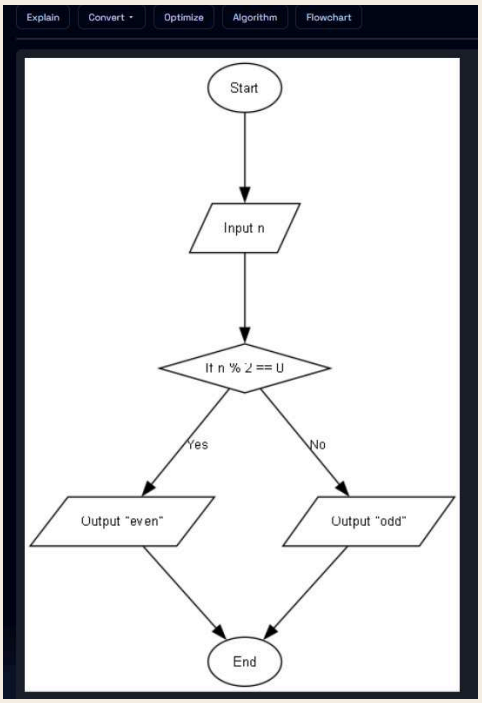


MODULE 5 - ALGORITHM & FLOWCHART GENERATION

MODULE 5

Flowchart generation





ASSUMPTIONS

- Availability of High-Quality External Knowledge Sources (e.g., GitHub repositories, GeeksforGeeks articles)
- Accurate Embedding Generation and Effective Retrieval Mechanism
- Stable and Reliable Internet Connectivity for accessing APIs and knowledge sources
- Input code is properly structured and Reasonably formatted for analysis

WORK BREAKDOWN

- Aleena Althaf - Module 5 (Algorithm & Flowchart Generation)
- Anithra Syindo - Module 3 (Code Conversion)
- Benita Benny - Module 1&2 (Syntax Checker & Optimization)
- Christine Michael - Module 4 (Explanation)
  -

SOFTWARE REQUIREMENTS FOR DEVELOPMENT

- Python – Core programming language used for backend logic and module implementation.
- Flask – Web framework used to build the backend server and handle requests.
- HTML, CSS, JavaScript – Used to develop the web-based user interface.
- ChromaDB – Vector database used to store and retrieve embeddings for the RAG module.
- Sentence Transformers (all-MiniLM-L6-v2) – Used to generate embeddings for code chunks.
- CodeT5-small – Used for code translation between Python and Java.
- Groq API with LLaMA 3.3 – Used for code explanation and algorithm generation.
- Graphviz – Used to generate and visualize flowcharts automatically.
- VS Code – Development environment used for coding and debugging.
- Python Libraries: AST, Javalang, Transformers, Torch

HARDWARE REQUIREMENTS FOR DEVELOPMENT

- Processor: Intel i5 / Ryzen 5 or equivalent
- RAM: 8 GB
- Storage: 10–20 GB free space
- Internet Connection: Required for API calls and retrieval
- Google Colab with T4 GPU (used for training CodeT5 small model)

SOFTWARE REQUIREMENTS FOR USER

- OS: Windows/Linux/macOS
- Browser: Chrome / Firefox / Edge
- Internet: Stable connection required
- Installation: None (web-based system)

DATASETS USED

- The dataset consists of Python–Java & Java–Python code pairs for source-level code translation.
- Each sample contains a Python/Java program as input and its equivalent Java/Python implementation as output.
- Approximately 2100+ code pairs were used for fine-tuning the CodeT5-small model.
- The dataset includes diverse programming constructs such as variable assignments, conditional statements, loops, functions, recursion, and basic data structures.
- The data was divided into training (90%) and validation/testing (10%) sets for model evaluation.

DATASETS USED

Java to Python

```
59  {"input": "task: java_to_python | code: import java.util.ArrayList;\nimport java.util.Arrays;\n\npublic class Main {\n    public static void main(String[] args) {\n
60  {"input": "task: java_to_python | code: public class Main {\n    public static void main(String[] args) {\n        for (int i = 0; i < 11; i++) {\n            System.ou
61  {"input": "task: java_to_python | code: public class Main {\n    public static void main(String[] args) {\n        String tempBuffer78 = \"Hello World\";\n        Syste
62  {"input": "task: java_to_python | code: public class Main {\n    public static void main(String[] args) {\n        int userScore20 = (69 * 50) / 2;\n        System.out.
63  {"input": "task: java_to_python | code: public class Main {\n    public static void main(String[] args) {\n        int resultMap18 = (67 * 75) / 2;\n        System.out.
64  {"input": "task: java_to_python | code: public class Main {\n    public static void main(String[] args) {\n        if (58 > 33) {\n            System.out.println(\"Grea
65  {"input": "task: java_to_python | code: public class Main {\n    public static int add(int a, int b) {\n        return a + b;\n    }\n    public static void main(String
66  {"input": "task: java_to_python | code: public class Main {\n    public static void main(String[] args) {\n        for (int i = 0; i < 7; i++) {\n            System.out
```

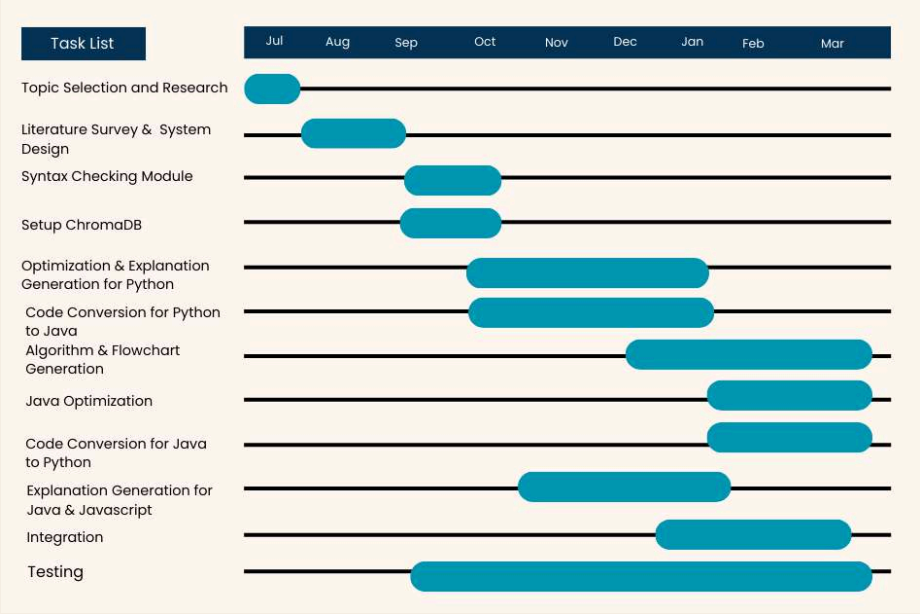
```
37 {\n        ArrayList<Integer> nums = new ArrayList<>(Arrays.asList(74, 29, 10));\n        for (int x : nums) {\n            if (x > 5) System.out.println(x);\n
 System.out.println(\"Iteration \" + i);\n        }\n    }\n}", "output": "for i in range(11):\n    print(f\"Iteration {i}\")"}
     System.out.println(tempBuffer78.toLowerCase().substring(0, 5));\n    }\n}", "output": "temp_buffer78 = \"Hello World\"\nprint(temp_buffer78.lower()[:5])"}
ystem.out.println(\"Result: \" + userScore20);\n    }\n}", "output": "user_score20 = (69 * 50) // 2\nprint(f\"Result: {user_score20}\")"}
ystem.out.println(\"Result: \" + resultMap18);\n    }\n}", "output": "result_map18 = (67 * 75) // 2\nprint(f\"Result: {result_map18}\")"}
tln(\"Greater\");\n        } else {\n            System.out.println(\"Smaller\");\n        }\n    }\n}", "output": "if 58 > 33:\n    print(\"Greater\")\nelse:\n    prin
ain(String[] args) {\n        System.out.println(add(53, 78));\n    }\n}", "output": "def add(a, b):\n    return a + b\n\nprint(add(53, 78))"}
System.out.println(\"Iteration \" + i);\n        }\n    }\n}", "output": "for i in range(7):\n    print(f\"Iteration {i}\")"}
```

Python to Java

```
5   {"input": "task: python_to_java | code: for i in range(3):\n    print(i)", "output": "public class Main {\n    public static void main(String[] args) {\n        for
6   {"input": "task: python_to_java | code: for i in range(4):\n    print(i)", "output": "public class Main {\n    public static void main(String[] args) {\n        for
7   {"input": "task: python_to_java | code: for i in range(8):\n    print(i)", "output": "public class Main {\n    public static void main(String[] args) {\n        for
8   {"input": "task: python_to_java | code: buffer41 = 53 + 3\nprint(buffer41 * 2)", "output": "public class Main {\n    public static void main(String[] args) {\n
9   {"input": "task: python_to_java | code: s = {1, 2, 3}\ns.add(4)\nprint(len(s))", "output": "public class Main {\n    public static void main(String[] args) {\n
10  {"input": "task: python_to_java | code: total83 = {'key': 10}\nprint(total83['key'])", "output": "import java.util.HashMap;\n\npublic class Main {\n    public stat
11  {"input": "task: python_to_java | code: for i in range(5):\n    print(i)", "output": "public class Main {\n    public static void main(String[] args) {\n        for
12  {"input": "task: python_to_java | code: for i in range(3):\n    print(i)", "output": "public class Main {\n    public static void main(String[] args) {\n        for
```

```
args) {\n        Set<Integer> s = new HashSet<>(Arrays.asList(1, 2, 3));\n        s.add(4);\n        System.out.println(s.size());\n    }\n} }
{\n        for (int i = 0; i < 3; i++) {\n            System.out.println(i);\n        }\n    }\n}"}
{\n        for (int i = 0; i < 4; i++) {\n            System.out.println(i);\n        }\n    }\n}"}
{\n        for (int i = 0; i < 8; i++) {\n            System.out.println(i);\n        }\n    }\n}"}
args) {\n        int buffer41 = 53 + 3;\n        System.out.println(buffer41 * 2);\n    }\n}"}
args) {\n        Set<Integer> s = new HashSet<>(Arrays.asList(1, 2, 3));\n        s.add(4);\n        System.out.println(s.size());\n    }\n}"}
  public static void main(String[] args) {\n        HashMap<String, Integer> total83 = new HashMap<>();\n        total83.put(\"key\", 10);\n        System.out.print
{\n        for (int i = 0; i < 5; i++) {\n            System.out.println(i);\n        }\n    }\n}"}
{\n        for (int i = 0; i < 3; i++) {\n            System.out.println(i);\n        }\n    }\n}"}
```

GANTT CHART



RISKS AND CHALLENGES

- Possibility of Inaccurate or Irrelevant Information Retrieval from the knowledge base
- Performance and Latency Issues due to API calls, embedding generation, and retrieval processes
- Dependence on Open-Source and Online Forum Content, which may vary in quality or accuracy
- Dynamic features, nested control flows, or unusual syntax can make analysis difficult.
- Ensuring that optimizations do not change the original program behavior.

CONCLUSION

- This project presents a unified AI-powered Code Assistant that integrates syntax checking, optimization, code conversion, visualization, and Retrieval-Augmented Generation (RAG) into a single platform.
- The system enhances code understanding by providing contextual explanations, detecting errors, convert, suggesting improvements, and generating algorithms and flowcharts.
- By combining AI models with retrieval techniques, the project aims to support students in learning, debugging, and improving code more efficiently.

REFERENCES

[1] S. Gao, C. Gao, W. Gu, and M. R. Lyu, "Search-Based LLMs for Code Optimization," in Proc. IEEE/ACM International Conference on Software Engineering, 2024.

[2] C. Zhang, Q. Zhou, M. Qiao, K. Tang, L. Xu, and F. Liu, "Re_Trans: Combined Retrieval and Transformer Model for Source Code Summarization," Entropy, vol. 24, no. 10, p. 1372, 2022.

[3] X. Yin, C. Ni, T. N. Nguyen, S. Wang, and X. Yang, "Rectifier: Code Translation with Corrector via LLMs," 2024.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

REFERENCES

[5] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Unifying Code Understanding and Generation via Pre-trained Models," Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 8696–8708, 2021.

[6] Y. Rong, T. Du, R. Li, and W. Bao, "Integrating LLM-based code optimization with human-like exclusionar reasoning for computational education," Journal of King Saud University - Computer and Information Sciences

THANK YOU

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis interpretation of data to provide valid conclusions.

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills
The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills
The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills
The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

Course Outcome 1: Model and solve real world problems by applying knowledge across domains (Cognitive knowledge level: Apply).

Course Outcome 2: Develop products, processes or technologies for sustainable and socially relevant applications (Cognitive knowledge level: Apply).

Course Outcome 3: Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks (Cognitive knowledge level: Apply).

Course Outcome 4: Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms (Cognitive knowledge level: Apply).

Course Outcome 5: Identify technology/research gaps and propose innovative/creative solutions (Cognitive knowledge level: Analyze).

Course Outcome 6: Organize and communicate technical and scientific findings effectively in written and oral forms (Cognitive knowledge level: Apply).

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES:

SNO	DESCRIPTION	BLOOM'S TAXONOMY LEVEL
1	Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant application.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND COURSE OUTCOMES (COs) – PROGRAM SPECIFIC OUTCOMES (PSOs)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	2	1		2	1	1	1	1	1	2	2	2
CO2	2	2	2		2	3	2	1	1	1	1	2	2	
CO3								3	2	2	1			3
CO4		1	2		3	2	3	2	2	3	1		2	2
CO5	2	3	3	1	2					1	3	3		
CO6			2			2	2	2	3	1	1			3

JUSTIFICATIONS FOR CO-PO MAPPING

Mapping	Low/Medium/ High	Justification
101003/CS822U.CO1-PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.
101003/CS822U.CO1-PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/CS822U.CO1-PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/CS822U.CO1-PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/CS822U.CO1-PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/CS822U.CO1-PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/CS822U.CO1-PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/CS822U.CO1-PO9	Low	Requires communication of results and findings through reports or presentations.
101003/CS822U.CO1-PO10	Low	Involves basic planning and execution of tasks while implementing solutions.
101003/CS822U.CO1-PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.
101003/CS822U.CO1-PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.
101003/CS822U.CO1-PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/CS822U.CO1-PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/CS822U.CO2-PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.

Mapping	Low/Medium/ High	Justification
101003/CS822U.CO2-PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/CS822U.CO2-PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/CS822U.CO2-PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/CS822U.CO2-PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/CS822U.CO2-PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.
101003/CS822U.CO2-PO8	Low	Development activities may require collaboration among team members.
101003/CS822U.CO2-PO9	Low	Involves communicating design ideas and development outcomes.
101003/CS822U.CO2-PO10	Low	Requires basic planning and coordination during development tasks.
101003/CS822U.CO2-PO11	Low	Encourages continuous learning of emerging technologies while building solutions.
101003/CS822U.CO2-PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/CS822U.CO2-PSO2	Medium	Uses programming and software development practices to implement solutions.
101003/CS822U.CO3-PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/CS822U.CO3-PO9	Medium	Requires effective communication among team members during collaboration.
101003/CS822U.CO3-PO10	Medium	Team leadership involves project coordination and task management.
101003/CS822U.CO3-PO11	Low	Team interaction supports learning from peers and adapting new approaches.
101003/CS822U.CO3-PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.
101003/CS822U.CO4-PO2	Low	Requires analysis and planning before executing project tasks.
101003/CS822U.CO4-PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/CS822U.CO4-PO5	High	Requires effective use of modern tools and resources during project execution.

Mapping	Low/Medium/ High	Justification
101003/CS822U.CO4-PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/CS822U.CO4-PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.
101003/CS822U.CO4-PO8	Medium	Task execution may involve teamwork and coordination among team members.
101003/CS822U.CO4-PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/CS822U.CO4-PO10	High	Focuses on project management including scheduling, resource allocation and task execution.
101003/CS822U.CO4-PO11	Low	Encourages adaptability and continuous learning during project implementation.
101003/CS822U.CO4-PSO2	Medium	Uses software development practices to execute and manage project tasks.
101003/CS822U.CO4-PSO3	Medium	Applies computer science knowledge in practical project development environments.
101003/CS822U.CO5-PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/CS822U.CO5-PO2	High	Requires deep problem analysis and identification of research gaps.
101003/CS822U.CO5-PO3	High	Proposes innovative design solutions to address identified gaps.
101003/CS822U.CO5-PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/CS822U.CO5-PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/CS822U.CO5-PO10	Low	Requires minimal planning while proposing new solution approaches.
101003/CS822U.CO5-PO11	High	Strongly promotes independent learning and exploration of emerging technologies.
101003/CS822U.CO5-PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.
101003/CS822U.CO6-PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/CS822U.CO6-PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/CS822U.CO6-PO7	Medium	Requires responsible and ethical communication of technical results.

Mapping	**Low/Medium/ High**	**Justification**
101003/CS822U.CO6-PO8	Medium	Communication supports collaboration within team environments.
101003/CS822U.CO6-PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/CS822U.CO6-PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/CS822U.CO6-PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/CS822U.CO6-PSO3	High	Communication of innovative computer science solutions and research outcomes.